Technology
Arts Sciences
TH Köln

Script zum Kurs

Systems on Programmable Chips

Tobias Krawutschke

Sommersemester 2026

Inhaltsverzeichnis

1 Einführung **4**
1.1 Learning Outcome . . . 4
1.2 Hardware-Plattform . . . 4
1.3 Software . . . 5
1.3.1 git . . . 5
1.3.2 VPN . . . 5
1.3.3 LaTeX . . . 5
1.3.4 Virtuelle Maschine . . . 6
1.4 Hardware Beschreibungssprache VHDL . . . 6
1.5 Ablauf und Durchführung der Lehrveranstaltung . . . 6
1.6 Git und Latex . . . 7

2 Tests und Testbenches **9**
2.1 Projektstruktur . . . 9
2.2 Hello Blinky . . . 9
2.2.1 Testbench . . . 9

3 Aufgabe: Umsetzung des Adressierungsbeispiels **10**
3.1 Address-Slave . . . 10
3.2 Address-Master . . . 10
3.3 Co-Simulation Master und Slave . . . 11
3.4 Synthese Master und Slave . . . 11
3.5 Metastabilität . . . 11
3.6 Ergebnisse im Portfolio . . . 11

4 Tests an der realen Hardware **13**
4.1 Synthese und Place and Route . . . 13
4.2 Aufgabe: einfacher UART-Test . . . 14

5 NEORV32 Einführung und Assembler **16**
5.1 Ordnerstruktur . . . 17
5.1.1 Eigene Programme kompilieren . . . 18
5.1.2 Programme Bare-Metal simulieren . . . 18
5.1.3 Programme Bare-Metal starten . . . 18
5.1.4 Programme über den Bootloader laden . . . 18
5.2 Aufgabe: Adressierungsarten und Alignement . . . 18
5.2.1 Kommentierter Assemblercode . . . 19
5.2.2 Analyse des Assemblercodes . . . 19

6 Hardware Erweiterung **20**
6.1 LED Ansteuerung . . . 20
6.1.1 Hardware Erweiterung . . . 20

6.1.2 software . . . 21
6.2 Aufgabe 7-Segment Dekoder . . . 22

7 WS2812 Display **23**
7.1 WS2812 Ansteuerung . . . 23
7.1.1 Aufgabe: Aufstellen der Formeln und Entwickeln des aserial Moduls . . . 23
7.2 Aus einem Speicher die WS2812 Matrix beschreiben . . . 24
7.2.1 Aufgabe: Vom ROM auf die WS2812 Matrix schreiben . . . 26
7.3 Dual-Port-RAM . . . 27
7.3.1 Aufgabe: Top-Level-Entity für DPRAM und CPU Integration . . . 28

8 Game Of Life **29**
8.1 GoL Software . . . 29
8.1.1 Kommunikationsprotokoll . . . 29
8.1.2 Aufgabe . . . 32

9 Implementierung der Kommunikation **33**
9.1 Aufgabe: UART auf dem NEORV nutzen . . . 34
9.2 Aufgabe: Implementierung des Kommunikationsprotokoll . . . 35
9.3 MMCP Protokoll Layer . . . 36
9.3.1 L1_receive . . . 36
9.3.2 L2_receive . . . 36
9.3.3 L3_receive . . . 36
9.3.4 L7_receive . . . 36
9.3.5 L7_send . . . 38
9.3.6 L3_send . . . 38
9.3.7 L2_send . . . 38
9.3.8 L1_send . . . 38
9.4 Test des Zugriffs . . . 39
9.5 ISO/OSI Referenzmodell . . . 40
9.5.1 Schichten, Protokolle, Dienste . . . 40
9.5.2 PDU, SDU, PCI, ICI . . . 40
9.5.3 Die OSI-Schichten . . . 41

A Iceduino Schaltplan **43**
A.1 Iceduino Schaltplan Ubersicht . . . 44
A.1.1 Power . . . 45
A.1.2 7-17V input Protection . . . 46
A.1.3 7-17V input 5V Regulator . . . 47
A.1.4 5V zu 1.2 V Regulator . . . 48
A.1.5 5V zu 3.3 V Regulator . . . 49
A.1.6 PLL Stromversorgung . . . 50
A.1.7 LEDs Taster Schalter . . . 51

A.1.8 SRAM . . . 52
A.1.9 USB Anschluss . . . 53
A.1.10 Level Shifter Header 50 MHz . . . 54
A.1.11 Arduino Header . . . 55
A.1.12 PMOD Header . . . 56
A.1.13 FTDI Programming Interface . . . 57
A.2 Iceduino Platine . . . 58
A.2.1 Kupfer Bestückungsseite . . . 58
A.2.2 Bestückungsdruck . . . 59

B Hello Blinky **60**

C Schaltplan Multi-Iceduino **63**

Abbildungsverzeichnis

1 gerenderte Ansicht iceduino . . . 4
2 Olimex-Board . . . 5
3 Zusammenhang zwischen Hardware- und Software Toolchain . . . 16
4 iceduino_setup Ordnerstruktur . . . 17
5 Daisy Chain der Datenleitungen der LEDs auf der Matrix-Platine . . . 24
6 Vorschlag Blockschaltbild (Prozesse oder Komponenten) der Ansteuerung der WS2812 LED Matrix . . . 25
7 Indizes der `inputs`-Matrix der 8 Bit breiten N-Eingänge . . . 30
8 Muster 1 . . . 32
9 Muster 2 . . . 32
10 Muster 3 . . . 32
11 Kommunikationskette am Beispiel von vier Teilnehmern . . . 33
12 MMCP Übersicht: Protokollfelder, Schichten und Längen, jedes Kästchen: 1 Byte . . . 34
13 Beispiel beteiligter Hard- und Software . . . 40
14 Schichten, Protokolle, Schnittstellen . . . 41
15 Zusammenhang zwischen PDU, SDU und PCI der Schichten . . . 41
16 TCP/IP über Ethernet, Darstellung eines Teils der OSI Layer . . . 42
17 Schaltplan des Base-Boards für mehrere Iceduinos . . . 63
18 Bestückungsplan des Base-Boards für mehrere Iceduinos . . . 64

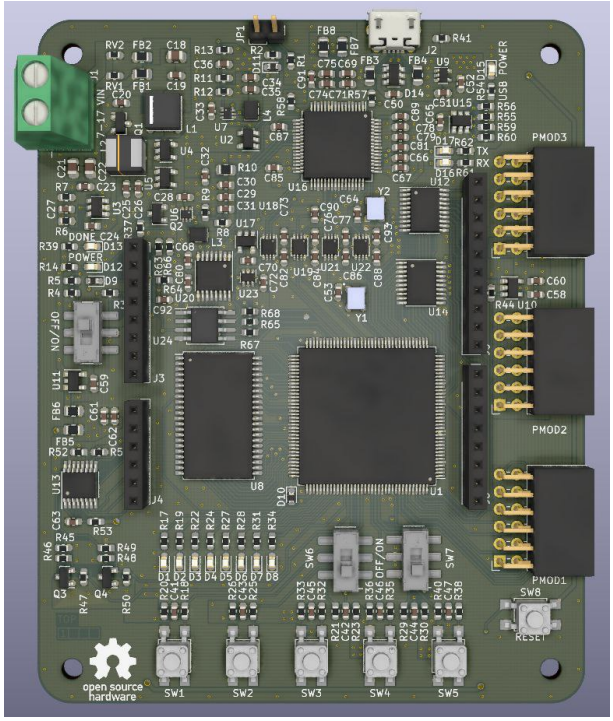


Abbildung 1: gerenderte Ansicht iceduino

1 Einführung

Der Kurs SoPC führt das Thema programmierbarer Hard- und Software anhand eines realen, projektbasierten Beipiels ein. Es beginnt bei selbst-erstellter Hardware. Das Erstellen einer Platine kann in diesem Kurs einigen Interessierten angeboten werden. Zum Test der erstellten Hardware werden FPGA-Konfigurationen erstellt, die unterschiedliche Baugruppen der Platine testen. Basis des SoPC bildet eine synthetisierbare Risc-V CPU (neorv32). Synthetisierbar heißt, dass die in VHDL vorliegende Beschreibung der CPU auf einen FPGA abgebildet werden kann und das entsprechende FPGA (hier ein Lattice HX4k) damit konfiguriert werden kann. Die auf dem RISC-V Prozessor ausführbare Software wird mit einem Cross-Compiler auf dem Host-System erstellt und auf das Board übertragen.

1.1 Learning Outcome

WAS: Die Studierenden entwerfen Hardware- und Software- Anteile eines „System on a Programmable Chip“ und entscheiden, welche Funktionalitäten in HW, welche in SW besser zu realisieren ist.

WOMIT: indem sie. . .

- Funktionalitäten und Ihre Anforderungen an HW/SW analysieren
- Hardwarebeschreibungssprachen wie VHDL anwenden
- Hardwarenahe Programmierung anwenden

WOZU: um später ein SoPC entwerfen zu können, das ein Spezialist unter eingebetteten Systemen ist, und eingesetzt wird, wenn besondere Anforderungen an Timing oder Datenmengen bestehen.

1.2 Hardware-Plattform

Das SoPC wird vorzugsweise auf dem iceduino-Board (s. Abbildung 1) umgesetzt. Alternativ wird das Olimex Board mit HX8K FPGA genutzt (s. Abbildung 2).

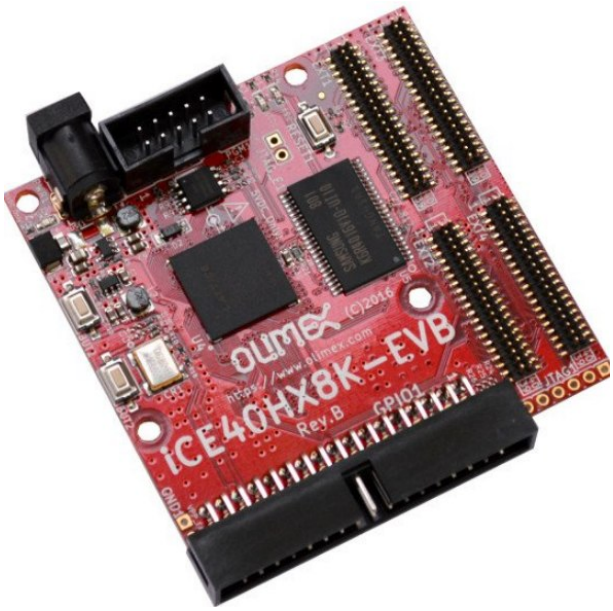


Abbildung 2: Olimex-Board [Bildquelle: `www.olimex.com`]

1.3 Software

Verschiedene Werkzeuge und Quellen werden für die Bearbeitung des Projekts benötigt.

1.3.1 git

Die Projektverzeichnisse der Gruppen und die Dokumentation finden sich auf dem gitlab Server des Labors für Digtialtechnik und Technische Informatik. Zum Auschecken, Bearbeiten und Hochladen wird mindestens das command-line tool git oder grafische Clients benötigt, z.B. `http://gitextensions.github.io/` oder

Noch neu bei git? Hier ein paar Infos/Tutorials zu git :

- `https://learngitbranching.js.org`: Interaktives Tutorial
- `https://gitimmersion.com/index.html`: Tutorial
- `https://git-scm.com/doc`: offizielle Git Docs
- `https://training.github.com/downloads/de/github-git-cheat-sheet/`: Cheatsheet, hilfreiche Einstiegsvideos

Bitte keine großen Binär-Dateien ins gitlab hochladen! Nutzen Sie die .gitignore Datei um temporäre build-Dateien nicht in Repository hochzuladen!

1.3.2 VPN

Der gitlab Server ist nur aus dem TH-Netz erreichbar, also von zu Hause aus über das TH-Köln VPN (Virtual Private Network)

1.3.3 LaTeX

Die komplette Dokumentation Ihres Lernfortschritts wird in LaTeX in diesem Dokument erfolgen. Sie erzeugen im git einen branch dieses Dokuments und ergänzen in den entsprechenden Kapiteln. Empfohlener Latex Editor: `https://texstudio.org/` (Installieren dauert ewig). Hier bitte empfehlenswerte Tutorials ergänzen.

1.3.4 Virtuelle Maschine

Wir haben eine VM erstellt, die unter Virtualbox läuft und bereits diverse Tools zur Hard-/Softwareentwicklung für dieses Projekt enthält. Bitte auch das Extension-Pack für den Zugriff auf das Host-USB installieren (wird benötigt, um die Boards zu programmieren). Die VM kann über sciebo `https://th-koeln.sciebo.de/s/Dffyrb7cL7xaqZW` heruntergeladen werden. Passwort: thkdr (Achtung! ein paar GiB) herunter.

Password des Nutzers thk: thkdr

Installierte Tools:

ghdl VHDL Simulationstool, übersetzt VHDL in simulierbaren Code

gtkwave Waveform Viewer zur grafischen Anzeige von Simulationen

yosys Open Source Synthese Tool zur Übersetzung von Verilog/VHDL[1]

nextpnr Open Source Place and Route Tool zur Abbildung der synhetisierten Netzliste auf entsprechende FPGAs (hier die Lattice HX1/4/8K Serie)

1.4 Hardware Beschreibungssprache VHDL

- Gesammelte VHDL Dokumentation der Uni Hamburg (Dort ist auch das *VHDL cookbook* von Peter J. Ashenden verlinkt)
- VHDL Aus der Mikrocontroller.net Artikelsammlung
- Lothar Miller (s. Kategorie VHDL)

1.5 Ablauf und Durchführung der Lehrveranstaltung

Im Verlauf der Veranstaltung werden folgende Fragestellungen und Themen anhand simulativer und praktischer Übungen bearbeitet:

- git / latex
- VHDL Grundlagen, Prozesse (kombinatorische, sequentielle)
- Testen / VHDL Testbenches
- Umsetzung komplexer Funktionalitäten mit Zustandsautomaten, Zählern und Hierarchien in VHDL
- Funktion dedizierter FPGA Hardware: z.B: Dual Port RAM, PLL
- Synthese, PCF Dateien
- serielle Schnittstelle: Pegel, Timing, Kodierung
- asynchrone Signale, Metastabile Zustände
- Abgrenzung Hardware / Software, Schnittstelle
- Risc-V CPU neorv32 Architektur und VHDL-Sourcen
- Speicher-Adressierung / -Alignement
- Memory mapped I/O read/write,- Umsetzung in Hardware
- Software zum Hardware Zugriff HAL, Treiber

[1] yosys und VHDL ist nur mit dem ghdl plugin möglich

- Hardware/ Software Partitionierung

Die dabei entstehenden Erkenntnisse anhand Aufgaben und Übungen werden in einem Portfolio dokumentiert, das jede Gruppe für sich als Latex Dokument im gitlab des Labors im Verlauf der Veranstaltung fortschreibt. Dort werden Übungsergebnisse anhand von erklärenden Texten, VHDL-Dateien, Simulationsverläufen und Fotos zusammengefasst und dienen bei der mündlichen Abschlussprüfung als Grundlage. Übungsaufgaben werden in der Regel im ILU zu vorgegebenen Terminen eingereicht (Abgabe des kompilierten Portfolios als pdf). Die Aufgaben sind kleinschrittig und werden in der Regel zum nächsten oder übernächsten Veranstaltungstermin umgesetzt.

Freitags ist vormittags (nach der Besprechung der bis dahin zu bearbeitenden Themen) wie auch nachmittags Zeit, die Aufgaben gemeinsam und unter Betreuung umzusetzen.

In der abschließenden Prüfung werden Fragen anhand des Portfolios besprochen. Sind Aufgaben zum Abgabetermin nicht bearbeitet oder nicht ausreichend dokumentiert, können die Themenbereiche nicht besprochen und bewertet werden, so das dadurch die maximal zu erreichende Note beschränkt wird. Eine Nachreichung der Ergebnisse ist bestimmt sinnvoll, wird aber die maximal zu erreichende Note nicht verbessern.

Als Beispiel: werden 20% der Aufgaben nicht bearbeitet, beschränkt sich die maximal zu erreichende Note auf eine 1,7. Das ULP ist bestanden, wenn 50% der Abgaben vorliegen.

1.6 Git und Latex

Für das Portfolio aus dem heraus auch die Quellen referenziert werden sollten nutzen Sie git und latex. Clonen Sie sich dieses Script vom gitlab Server und nutzen ihn als Basis für das eigene Portfolio in Ihrem gitlab-Projekt.

Um VHDL Code mit Syntax-Highlighting abzudrucken ist das minted Paket notwendig, das funktioniert aber nicht direkt. [2] Der folgende durch die *verbatim* Umgebung auskommentierte Code kann in der Regel nicht übersetzt werden:

```
\begin{code}
 \captionof{listing}{VHDL Hello World}
 \label{code:hellovhd}
 \inputminted[linenos,frame=single,fontsize=\footnotesize]{vhdl}{vhdl/hellovhd.vhd}
\end{code}

\begin{minted}{vhdl}
f <= '0' when (c = '0') and (b = '1') and (a = '0') else -- m2
     '0' when (c = '1') and (b = '1') and (a = '0') else -- m6
     '1';
\end{minted}
```

So sieht das dann aus:

Listing 1: VHDL Hello World

```vhdl
entity HelloWorldTb is
end entity;

architecture sim of HelloWorldTb is
begin

    process is
    begin
        report "Hello World!";
        wait;
    end process;

end architecture;
```

```vhdl
f <= '0' when (c = '0') and (b = '1') and (a = '0') else -- m2
```

[2] siehe hier `SOPC2026_Script.tex`, Zeile 13: \usepackage{minted}

```
'0' when (c = '1') and (b = '1') and (a = '0') else -- m6
'1';
```

Installieren Sie minted und die Integration in den Editor, so dass hier VHDL-Code integriert werden kann. Die Voraussetzung für das Package minted ist **Python** in der Version 3.10 oder neuer (stable). Im speziellen benutzt minted das Python-Script **Pygments**, das mit Hilfe des Pip-Befehls von der Konsole aus installiert werden kann.

In der LaTeX-Distribution TeXstudio wird minted bei der Installation mitgeliefert, jedoch ist eine Anpassung in den Optionen erforderlich. Hierbei muss man in *Options > Configure TeXstudio > Commands* gehen und am Parameter *PdfLaTeX* zusätzlich den Flag -shell-escape am Ende anhängen:

```
pdflatex.exe -synctex=1 -interaction=nonstopmode -shell-escape %.tex
```

Um minted jetzt in LaTeX zu verwenden, sind zwei Befehle sehr hilfreich:

```
> \usepackage{minted}: für das Initialisieren von minted,
> \begin{minted}{LANG} und \end{minted}: für das markieren des Codes,
dabei wird "LANG" durch die Programmiersprache ausgetauscht, z.B. vhdl.
```

Bei der Kombination git und LaTeX empfiehlt es sich jeden Satz in eine eigene Zeile zu schreiben. Damit können Änderungen am gleichen Dokument einfacher nachverfolgt werden, da Unterschiede (*diffs*) pro Zeile nachvollzogen werden. Einfache Zeilenumbrüche werden von LaTeX ignoriert und zu einem Absatz zusammengefügt. Erst zwei Zeilenumbrüche führen zu einem Abschnittswechsel.

2 Tests und Testbenches

2.1 Projektstruktur

Um eine Simulation mit einer einfachen Testbench zu betrachten und eine zukünftige Synthese vorzubereiten, erstellen wir zunächst eine typische Ordnerstruktur:

- `rtl`
- `sim`
- `syn`

Der eigentliche Code kommt in den Ordner `rtl` (für Register Transfer Level), da hinein kommt der synthetisierbare Code, der auch auf FPGAs / ASICs abgebildet werden kann. Der `sim` Ordner enthält die Testbench oder Testbenches und besteht größtenteils aus nicht-synthetisierbarem Code. Für die Synthese sind in der Regel Zusatzdateien notwendig, z.B. Constraints-Files (pcf), die angeben welches Signal auf welchen Pin gelegt wird, oder welche Taktrate maximal am clock-Eingang anliegt. Diese Dateien kommen in `syn`.

2.2 Hello Blinky

Der Beispielcode dafür findet sich im Anhang Hello Blinky. In den Unterverzeichnissen der LaTeX Sourcen zu diesem Script findet sich die komplette Ordnerstruktur, die so auch mit dem iceduino Board getestet werden kann.

2.2.1 Testbench

Die Testbench ist in der Hierarchie über dem zu untersuchenden Modell (DUT,- Device Under Test) angesiedelt und hat selber keine Ein- und Ausgänge, was an einer leeren Entity-Deklaration ersichtlich ist. Das DUT wird in der Testbench instanziiert und die Testbench übernimmt dann die Aufgabe der Umgebung in der der FPGA arbeitet. Z.B. befinden sich auf dem iceduino Board Oszillatoren (12 MHz und 50 MHz), die den FPGA mit Takt versorgen. Dieser sogenannte *Stimulus* wird in der Testbench erzeugt. Dabei werden einfach VHDL Ausdrücke genutzt, um ein Verhalten zu spezifizieren, dass simuliert, aber nicht synthetisiert werden kann:

```
Clk <= not Clk after 10 ns;
```

Es gibt keine einfachen digitaltechnischen Elemente und somit auch keine FPGA Ressourcen, die eine spezifizierte Verzögerungszeit umsetzen könnten.

Weitere typische Stimuli, wie beispielsweise reset Signale und Eingangssignale werden ebenfalls in einfacher, nicht synthetisierbarer Form, in der Testbench erzeugt.

Das Verhalten des simulierten Modells kann dann in einem Waveform-Viewer wie *gtkwave* beobachtet werden. Ein regelgerechter Test wird mit *Assertions* umgesetzt. Dabei erzeugt der Simulator Ausgaben, je nach dem, ob das untersuchte Signal den angenommenen Wert besitzt, oder nicht. Im hello_blinky Beispiel wird untersucht, ob der LED-Ausgang nach einer bestimmten Zeit 0 bzw. 1 ist. Über das Schlüsselwort *severity* kann der loglevel der Ausgabe eingestellt werden, vom informierenden Level *note* über *warning* bis zu *error*.

3 Aufgabe: Umsetzung des Adressierungsbeispiels

Anhand des Adressierungsbeispiels soll an einem einfachen Modell Adressierung umgesetzt werden und in Simulation und an der Hardware umgesetzt werden. Die Hauptaufgabe ist hier die Dokumentation des Entwicklungsprozesses und eine Untersuchung des metastabilen Zustands von Flip-Flops.

3.1 Address-Slave

In dem umzusetzenden Beispiel sollen die 8 LEDs des iceduino Boards adressierbar angesprochen werden können. Mit drei Adressleitungen kann jede einzelne LED ausgewählt werden (dual 000 bis 111) und über ein *write enable* (WE) Signal mit Daten (D) beschrieben werden können.

Solange WE nicht aktiv (0) ist, wird der aktuelle Zustand gespeichert und jegliche Änderung an Adresse und Daten Eingang ignoriert.

Ist WE aktiv (1) wird das Datenbit, das am Eingang anliegt, bei der nächsten positiven Taktflanke in das LED-Register übernommen. Beispiele:

- we=1, d=1, adr=110 : LED 6 wird eingeschaltet
- we=1, d=0, adr=000 : LED 0 wird ausgeschaltet
- we=0, d=x, adr=101 : keine Änderung

Setzen Sie den Slave mit Takt (50 MHz Eingang) und Reset-Eingang basierend auf der hello-blinky Struktur (rtl,sim,syn) um und erstellen eine Testbench, die drei ausgewählte Schreibzyklen testet und die gewünschte Funktion mit asserts bestätigt.

3.2 Address-Master

Der Master soll die Funktion der Testbench übernehmen, aber auch synthetisierbar werden, um zusätzlich im FPGA ausgeführt werden zu können.

Der Master soll den Adressbereich von 0 bis 7 in der Art durchgehen, dass die Funktion vom menschlichen Betrachter nachvollzogen werden kann: Mit einer Verzögerungszeit von 1/4 Sekunde soll eine Art Lauflicht dargestellt werden. Folgender Ablauf ist denkbar:

- adressierte LED einschalten
- 1/4 Sekunde warten
- adressierte LED ausschalten
- Adresse erhöhen (mit overflow-wrap)
- Beginn von vorn

Dafür bietet sich eine Kombination aus folgenden digitaltechnischen Grund-Module an:

- synchroner Taktteiler: erzeugt alle 1/4 Sekunde ein Signal für das Abwarten
- Zähler für die Adressen 000 bis 111
- Zustandsautomat zur Kontrolle der zuvor genannten Module

Erstellen Sie ein Blockschaltbild und ein Zustandsüberführungsdiagramm. Erstellen Sie den VHDL Code.

3.3 Co-Simulation Master und Slave

Verbinden Sie Master und Slave Code in einem übergeordneten Entity *adressing.vhd*:

```
entity addressing is
  port (
    clk_12MHz : in  std_logic; -- zum adr_slave
    clk_50MHz : in  std_logic; -- zum adr_master
    leds      : out std_logic_vector(7 downto 0)  -- LED Ausgänge Slave
    );
end addressing;
```

Dort sollte auch noch der Power-On-Reset Generator eingefügt werden:[3]:

```
-- async_reset als std_logic Signal anlegen!
process(clk_12Mhz) is
  variable cnt : unsigned(7 downto 0) := (others => '0');
begin
  if rising_edge(clk_12Mhz) then
    if cnt < 255 then
      cnt := cnt + 1;
      async_reset <= '1';
    else
      async_reset <= '0';
    end if;
  end if;
end process;
```

Die übergeordnete Simulation braucht jetzt nur noch die beiden Taktquellen zu erzeugen. Finden Sie eine Lösung mit *generics* und *generic map*, um für die Simulation eine kürzere Wartezeit umzusetzen, als für die Synthese, um die Simulationszeit und zu verkürzen und keine riesigen Simulationsdateien entstehen zu lassen.

3.4 Synthese Master und Slave

Für die Synthese kann die auf hello_blinky basierende pcf Datei für die erweitert werden. Die 8 Bits des Ausgangsvektors zu den LEDs müssen einzeln den Pin-Nummern zugewiesen werden:

```
# PCF File adressing.pcf
set_io clk_12MHz 21    # 12 MHz: Pin 21
set_io clk_50MHz 52    ä 50 MHz: Pin 52
# LEDs
set_io leds[0] 112
set_io leds[1] 113
...
```

3.5 Metastabilität

Im Realtest werden ab-und-zu Aussetzer beobachtet, die auf Metastabilität zurückzuführen sind. Nennen Sie Gründe, wie Flip Flops in metastabile Zustände kommen und welche Auswirkung das hat und wie das verhindert werden kann. Sie dazu auch den hervorragenden Artikel von Lothar Miller: `http://www.lothar-miller.de/s9y/categories/35-Einsynchronisieren`

3.6 Ergebnisse im Portfolio

Stellen Sie in Ihrem Portfolio das Adressierungsprojekt vor und strukturieren Sie es sinnvoll:

[3] Der POR-Reset Generator erzeugt den Einschalt Reset als Signal nach der Neukonfiguration des FPGA, welche durch drücken des Reset-Tasters ausgelöst wird. Ein direktes Reset-Signal liegt nicht als Eingang vor, deshalb dieser „Trick“

- Slave Code und erläuternder Text zur Funktion
- Slave Testbench Code und erläuternder Text zur Funktion, erwähnen Sie asserts und nicht-Synthetisierbarkeit von Testbench-Code
- Simulation Slave mit Testbench (Screenshot + Was sehen wir da? Ursache Wirkung)
- Master Blockschaltbild incl. Slave-Anbindung
- Master Code
- Master Simulation mit Slave
- Erläutern Sie die Simulationsergebnisse: Master Ablauf der Statemachine
- Nennen Sie Gründe, wie Flip Flops in Metastabile Zustände kommen und welche Auswirkung das hat und wie das verhindert werden kann

4 Tests an der realen Hardware

Der nächste Schritt ist die Umsetzung auf dem iceduino Board. Zunächst mit dem Adressierungs-Beispiel, dann mit einer UART Umsetzung.

4.1 Synthese und Place and Route

Um den in rtl enthaltenen VHDL Code in eine binäre Datei zu übersetzen, mit dem der FPGA auf dem iceduino Board konfiguriert werden kann, sind mehrere Schritte notwendig:

- Synthese mit yosys, s. Abschnitt *compile* im Makefile. Yosys[4] kompiliert den Sourcecode in eine Netzliste (hier im json Format). Yosys erwartet in der Regel Verilog, aber durch ein GHDL plugin ist die Verarbeitung von VHDL möglich.
- Place and Route mit nextpnr: Die erzeugte Netzliste wird auf den gewählten FPGA abgebildet und dabei physikalische Vorgaben (Physical Design Constraints, PCF) angewendet. In der Regel werden die zu erreichende Taktrate (f_{max}) vorgegeben und die Pins, auf die die VHDL-Signale gelegt werden sollen.
- Wandel in ein Programmierformat (in diesem Fall mit *icepack* und *icestorm*, dedizierte open-source Tools für die eingesetzten Lattice FPGAs)

Das Makefile in Listing 2 stellt einen einfachen Aufruf der Kompilierkette dar.

Listing 2: Makefile Synthese

```
# vhdl files
TOPLEVEL=hello_blinky
FILES = ../rtl/*vhd
VHDLEX = .vhd

# Physical Constraint File
PCFFILE=hello_blinky.pcf

#GHDL CONFIG
GHDL_CMD = ghdl
GHDL_FLAGS  =

#YOSYS CONFIG
YOSYS_CMD = yosys

#NEXTPNR CONFIG
NEXTPNR_CMD = nextpnr-ice40
PACKAGE = tq144:4k
NEXTPNR_OPT = --freq 50 --hx8k --package $(PACKAGE)

SYNDIR = synthesis
STOP_TIME = 10us
GHDL_SIM_OPT = --stop-time=$(STOP_TIME) --stop-delta=100000

.PHONY: clean

all: clean binary


compile:
        @mkdir -p $(SYNDIR)
        @$(GHDL_CMD) -i $(GHDL_FLAGS) --workdir=$(SYNDIR) --work=work $(FILES)
        @$(GHDL_CMD) -m $(GHDL_FLAGS) --workdir=$(SYNDIR) --work=work $(TOPLEVEL)


synthesize: compile
        @$(YOSYS_CMD) -p 'ghdl --workdir='$(SYNDIR)' '$(TOPLEVEL)'; synth_ice40 -json '$(SYNDIR)/$(TOPLEVEL)'.json'

placeandroute: synthesize
        $(NEXTPNR_CMD) $(NEXTPNR_OPT)  --pcf $(PCFFILE) --asc $(SYNDIR)/$(TOPLEVEL).asc --json $(SYNDIR)/$(TOPLEVEL).json

binary: placeandroute
        @icepack $(SYNDIR)/$(TOPLEVEL).asc $(SYNDIR)/$(TOPLEVEL).bin
```

[4] YOSYS steht für „**Y**osys **O**pen **SY**nthesis **S**uite“ und ist somit ein rekursives Akronym wie z.B. GNU

```
44
45 program: $(SYNDIR)/$(TOPLEVEL).bin
46         @iceprog $(SYNDIR)/$(TOPLEVEL).bin
47
48 clean:
49         @rm -rf $(SYNDIR)
```

Zu beachten ist die Vorgabe der Physical Design Constraint als Datei in Zeile 7: PCFFILE und weitere Design Constraints als Kommandozeilenparameter für nextpnr in den NEXTPNR_OPT in Zeile 19.

Die referenzierte Datei könnte im Minimalfall so aussehen:

Listing 3: Minimale PCF Datei

```
1 #set_io clk 21    # 12 MHz: Pin 21
2 set_io clk 52    # 50MHz: Pin 52
3 set_io reset 122 # button 1
4
5 set_io led   112 # LED0
```

Zeile 1 ist auskommentiert, so dass für das clk Signal der Pin 52, an dem der Ausgang des 50 MHz Oszillators angeschlossen ist. Pins sind im Schaltplan dokumentiert. Als Beispiel sei hier die Quelle des Takts vorgestellt: Auf der Seite Level Shifter Header 50 MHz findet sich das Signal *IC_CLK_50M*, das auf der Seite Iceduino Schaltplan Ubersicht in die Bank 2 an Pin 52 angelegt ist. Der 12 MHz Oszillator findet sich unten links auf der Seite FTDI Programming Interface des Schaltplans.

Der **Reset** im Schaltplan ist an Pin 66 angeschlossen. Es ist der Device-Reset und löst eine Neu-Konfiguration des FPGAs aus. Dabei wird aus dem Flash Baustein die FPGA Konfiguration neu geladen und Register mit Ihrem Initialisierungswert geladen (:=) und der Chip neugestartet. Aus der „Fabric“ (so wird das Gate-Array auch genannt) ist kein Zugriff auf diesen Reset-Pin möglich und kann also nicht für einen Eingangspin als reset genutzt werden. Entweder nutzt man einen der externen Taster, oder den hier vorgestellten workaround, der so im neorv32 Prozessor genutzt wird und ein ca. 5 us langes Einschaltresetsignal erzeugt.

```
process(clk_50mhz) is
  variable cnt : unsigned(7 downto 0) := (others => '0');
  begin
    if rising_edge(clk_50mhz) then
      if cnt < 255 then
        cnt := cnt + 1;
        external_rstn <= '0';
      else
        external_rstn <= '1';
      end if;
    end if;
  end process;
```

Das funktioniert, da die Initialisierung der Variable cnt mit 0 in der Konfiguration steht. Nach dem Ablauf der reset-Zeit wird cnt nicht mehr zurückgesetzt. Dies ist nur über eine Neu-Konfiguration (Device-Reset) möglich.

4.2 Aufgabe: einfacher UART-Test

Wenn das iceduino Board mit dem PC verbunden ist, werden über den FTDI Chip (s. auch FTDI Programming Interface auf Seite 57) zwei Kanäle bereitgestellt: Der erste Kanal ist zur Programmierung des Flash Chips, der die FPGA Konfiguration enthält. Beim POR (Power-On-Reset) wird der FPGA aus dem Flash konfiguriert. Der zweite Kanal des FTDI Chips meldet sich als regulärer UART am PC an (z.B /dev/ttyUSB5 unter Linux oder COM7 unter Windows). Die FTDI Pins BDBUS0 (FTDI_TXD) und BDBUS1 (FTDI_RXD) sind über Pegelwandler mit den Pins *TXD*(60) und *RXD*(62) des FPGAs verbunden. (s. U1D auf Seite 44). Beachten Sie, dass an dieser Stelle TX des FTDI zum RX des FPGAs wird. An diese Pins können zwei (theoretisch) voneinander unabhängige Protokollmaschinen jeweils für das Empfangen und das Senden des UART Protokolls implementiert werden. An diesen Pins liegt das RS-232 Protokoll im TTL Pegel (also nicht nach RS-232 [5] Standard) an.

[5] s. auch Abschnitt *Signalpegel* auf `https://www.mikrocontroller.net/articles/RS-232`

Erstellen und Testen Sie

- TX-Test: Das Board sendet ein Zeichen oder Zeichenkette in 9600 Baud 8N1, wenn ein Taster gedrückt wird.
- RX-Test mit 9600 8N1: Schalte LED an, wenn Zeichen x empfangen wird, aus wenn Zeichen y empfangen wird.

Dokumentieren Sie in Ihrem Portfolio:

- Aufgabenstellung
- jeweils Blockschaltbilder der entworfenen Hardware für RX/TX mit Beschreibung
- VHDL Code der RX und TX Systeme
- Erläuternden Text, der Blockschaltbilder und Codes in Bezug setzt
- Testbench für den RX- und TX-Teil inklusive Assert Tests zu
  - TX: den Sendebits nach Tastendruck
  - RX: LED 1x An, 1x Aus nach Empfang der jeweiligen Zeichen
- Erläuterung der Testbenches und der erwarteten Ausgabe der Asserts

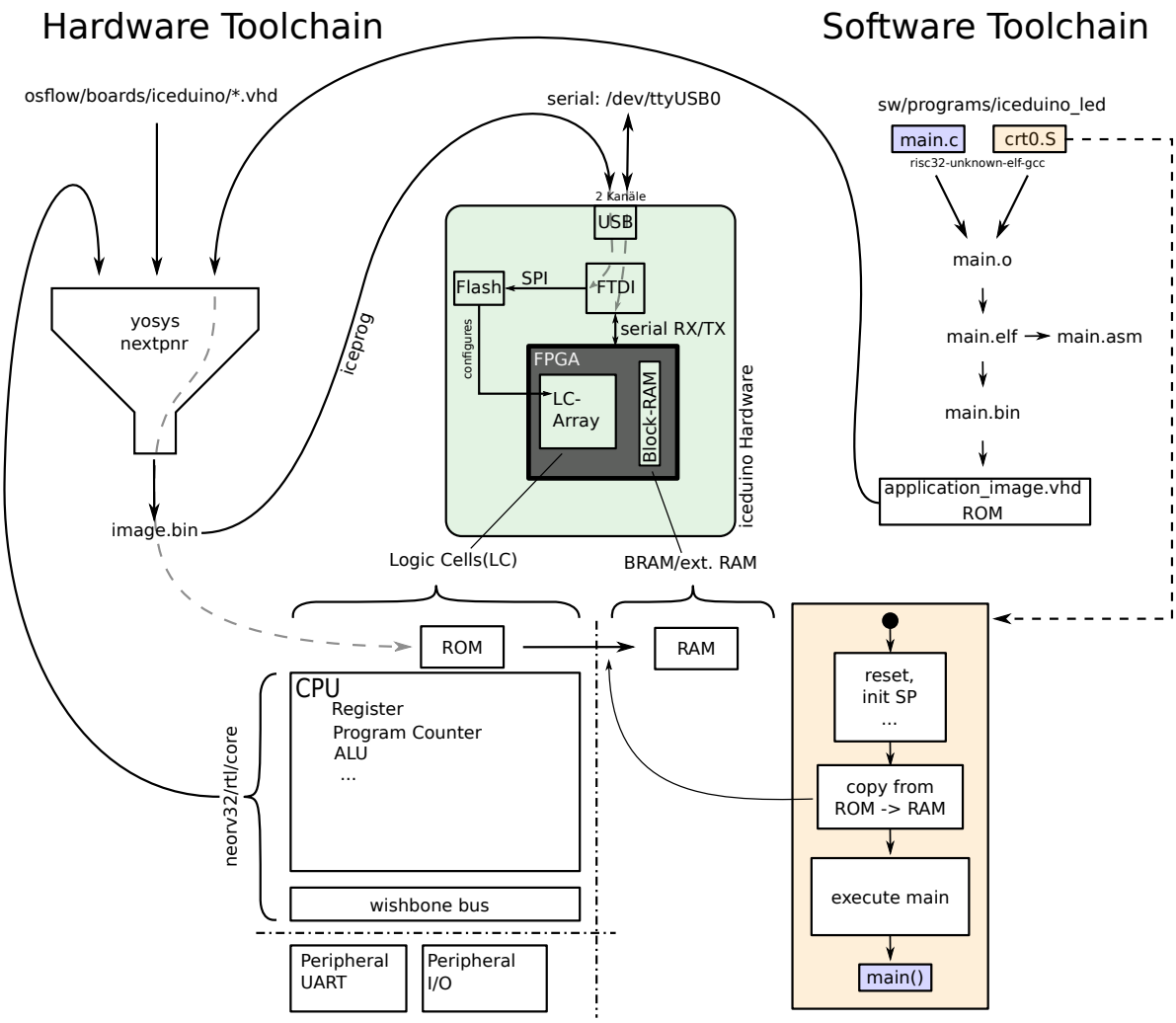


Abbildung 3: Zusammenhang zwischen Hardware- und Software Toolchain

5 NEORV32 Einführung und Assembler

Dieses erste Beispiel basiert auf dem NEORV32 Sourcecode, der für das iceduino Board angepasst wurde. Auf der VM (Version >=1.7)liegt ein Desktop Ordner `iceduino_setup`. Sollte Ihre noch Version 1.6 oder davor sein, bitte über den ILU-Link herunterladen.

Das Zusammenspiel der verschiedenen Quellcode-Dateien für Software und Hardware, sowie die dafür genutzten Toolchains ist in Abbildung 3 illustriert: Der Hardware-Quellcode bestehend aus verschiedenen *.vhd Dateien beschreibt die RiscV-CPU mit ihrer internen Struktur bestehend aus Registern, Programmzähler, ALU, etc., den on-chip Bussen (wishbone) und der Peripherie, wie UART und I/O. Auch ROM (Read Only Memory) wird über ein VHDL-Array beschrieben (s. application_image.vhd). Internes RAM (Random Access Memory, Lesen und Schreiben) wird zwar auch über VHDL beschrieben, jedoch im Gegensatz zu den zuvor genannten Modulen nicht durch logische Zellen (LC=logic Cells) umgesetzt. Erkennt das Synthese Tool, dass eine in VHDL beschriebene Funktion mit RAM umsetzbar ist, wird das im FPGA Baustein vorliegende RAM eingesetzt.

Die Hardware Toolchain, bestehend aus yosys, nextpnr und weiteren Tools erzeugt eine Programmierdatei, in der Abbildung 3 generisch *image.bin* genannt, die mit dem Programmiertool *iceprog* in den Flash-Baustein auf dem Board übertragen und nicht-flüchtig gespeichert werden kann. Nach einem Reset oder Power-Cycle lädt der FPGA-Baustein die Konfiguration der logischen Zellen aus dem Flash-Baustein und konfiguriert seine Funktionalität. BRAM kann in der Regel nicht vorkonfiguriert werden. Der Speicherinhalt ist nach dem Einschalten oder Reset leer oder zufällig gefüllt.

Wenn, wie in diesem Fall eine CPU im FPGA konfiguriert ist, gibt es je nach Typ bestimmte Vorgehen, um den nach einem Reset auszuführenden Code in den RAM zu bekommen.

Dazu wird zunächst die Software-Toolchain betrachtet: Wie in Abbildung 3 zu sehen wird der *main*-Code zusammen mit einer Inititalisierungsfunktion *crt0.S* kompiliert. Dies geschieht in der VM mit einem

Abbildung 4: iceduino_setup Ordnerstruktur

cross-compiler, also ein Compiler der nicht Code für das eigene System, sondern für ein anderes System erzeugt. In diesem Fall für den Risc-V Prozessor.

Eine der Aufgaben des *crt0.S* Assemblercodes ist es, neben notwendigen Initialisierungen der CPU und Peripherie den Programmcode vom ROM ins RAM zu kopieren. Der Code von *crt0.S* wird direkt aus dem ROM heraus ausgeführt. Nach dem Kopieren wird auf die *main()*-Funktion im RAM verzweigt.

Um den kompilierten Binärcode bestehend aus *crt0.S* und *main* als ROM in das VHDL-Modell der CPU zu bekommen wird der kompilierte Code als VHDL-Array im application_image.vhd abgelegt und in den Hardware-Toolchain Teil kopiert. Dort wird er dann zusammen mit dem Rest der Hardware synthetisiert und über iceprog im Flash-Baustein für die Konfiguration des FPGAs abgelegt.

In der Regel enthält diese hard-codierte Software einen Bootloader, der dann aus anderen Speichermedien oder der seriellen Schnittstelle weiteren Code nachlädt, der dann ausgeführt wird. Unser erstes Beispiel ist bare-metal (kein Bootloader, kein Betriebssystem) umgesetzt und enthält eine Software, um die LEDs Blinken zu lassen.

Einige Ressourcen zur NEORV32 CPU und RISC-V Assembler:

- NEORV32 Datasheet
- NEORV32 User Guide
- RISC-V Homepage, RISC-V Instruction Set Architecture (ISA) Spezifikationen
- RISC-V Reference
- RISC-V Assembly for Beginners

5.1 Ordnerstruktur

Der Iceduino NEORV32 ist in 4 Bereiche eingeteilt:

- **neorv32**: Hier liegt das Repository des NEORV32. Es werden hauptsächlich die VHDL-Sourcen und das Software Framework für den NEORV32 genutzt.
- **osflow**: Dieser Ordner beinhaltet die Konfiguration für den Iceduino.
  - **osflow/boards/iceduino**: HIer liegen die VHDL Sourcen für den Iceduino, unsere spezielle Implementation des NEORV32 auf diesem Board
  - **osflow/constraint**: Enthält die .pcf Datei (physical constraints file). Hier werden die Ports der top-VHDL Datei den physischen Pins des FPGA zugewiesen.
  - **osflow/device/ice40**: Enthält wrapper Komponenten für den ice40 für yosys (SB_HFOSC, SB_PLL40_CORE,...)
- **simulation**: Beinhaltet eine Testbench für den Iceduino. Simuliert wird mit GHDL und GTKWave.
- **sw**: Dieser Ordner beinhaltet weiter Ordner, die das Software-Framework für den Iceduino darstellen:
  - **sw/bootloader**: Der Bootloader liegt als .c Datei vor und kann neu kompiliert werden. Das binäre Artefakt wird in ein VHDL-Array (`neorv32_bootloader_image.vhd`) gewandelt und mit der Hardware zusammen kompiliert.

- **sw/common**: Enthält die Assembler-startup Datei *crt0.S*, das Linkerskript in dem die Speicheraufteilung definiert wird, sowie common.mk, mit den gemeinsam genutzten Makefile-Zielen und Definitionen.
- **sw/lib**: Beinhaltet die Bibliothek für den Iceduino.
- **sw/programs**: Hauptordner für die Programme für den Iceduino.

5.1.1 Eigene Programme kompilieren

1. Einen Ordner unter **sw/programs**/ erstellen.
2. Das Makefile aus dem Besipiel in den neuen Ordner kopieren.
3. `main.c` erstellen und schreiben.
4. Ein Terminal (interaktive shell) in dem Ordner öffnen
5. Das VHDL-Image `neorv32_application_image.vhd` (also ein VHDL-Array, das die binäre Darstellung des übersetzten Programms enthält) wird mit „make clean install“ erzeugt und an den Ort osflow/boards/iceduino kopiert.

5.1.2 Programme Bare-Metal simulieren

In `osflow/boards/iceduino/neorv32_iceduino_top.vhd` das generic:

`INT_BOOTLOADER_EN => false` setzen.

Im Ordner **simulation**/ zunächst die Simulationsdauer überprüfen (s. Makefile `--stop-time="..."`), mit „make all“ die Simulationsdateien erzeugen und mit „make wave“ die Simulation und GTKWave starten.

5.1.3 Programme Bare-Metal starten

In `osflow/boards/iceduino/` „make“ eingeben (Ausführung von Synthese und Place and Route). Um die FTDI-Schnittstelle des angesteckten Boards an die VM weiterzugeben im Menü von VirtualBox unter Geräte/USB den *FTDI Dual RS-232 HS* auswählen. Das Programmieren des Flash Speichers wird danach mit „make program“ gestartet (Dauer ca. 2..3 Minuten)

Siehe auch Kapitel 6 des NEORV32 User Guide

5.1.4 Programme über den Bootloader laden

Siehe Kapitel 5 des NEORV32 User Guide

5.2 Aufgabe: Adressierungsarten und Alignement

Übersetzen Sie folgendes C-Programm für den NEORV32 Prozessor:

Listing 4: Test der Speicherzugriffs-Arten

```c
#include <neorv32_iceduino.h>

#define COUNT_VALUES 120

uint8_t Values[COUNT_VALUES];
uint32_t* value_ad1, *value_ad2, *value_ad3;
uint32_t value1, value2, value3;
int main() {
            for(uint8_t count = 0; count < COUNT_VALUES; count++)
                    Values[count] = count;
            value_ad1 = (uint32_t*)(&(Values[0]));
            value_ad2 = (uint32_t*)(&(Values[8+2]));
            value_ad3 = (uint32_t*)(&(Values[12+1]));
            value1 = *(value_ad1);
```

```
15            value2 = *(value_ad2);
16            value3 = *(value_ad3);
17
18            while(1){
19            }
20
21    return 0;
22  }
23
```

Ein uint8_t array wird erzeugt und mit Werten gefüllt (Z.9/10). In den Zeilen 11-13 werden 3 uint32_t Pointer `value_adX` angelegt und mit verschiedenen Adressen des uint8_t Arrays initialisiert (0,10 und 13). Danach folgen 3 Lesezugriffe von dem Array. Interpretieren Sie den Assembler Code (Tipp: Der Makefile-Variablen USER_FLAGS in *common.mk* ein „-g“ hinzufügen. Dazu beobachten Sie zusätzlich die Ausführung des Codes in einer Simulation. Trotz getrennter Daten- und Instruktions- zugriffen der CPU handelt es sich um eine von-Neumann Architektur, da die Zugriffe in die CPU in Bus-Multiplexer vereint werden. Harvard-Architektur hat einen getrennten Daten und Instruktions-Bus (s. auch `https://stnolting.github.io/neorv32/#_address_space`)

Hilfreich dabei ist die Datei `address_access.gtkw` (umbenennen! ILU macht daraus *address_access-gtkw.sec*), die neben den drei Speicherbereichen der Variablen value1, value2 und value3 auch die Registerinhalte anzeigt. [6]

Schauen Sie sich den Assemblercode an und simulieren den Code Zum Finden der Assembler-Instruktionen, bei denen value zugewiesen wird, müssen Sie erst die crt0.S durchlaufen lassen (ca. 250 us). Die Schleife zum Initialisieren des 8-Bit Arrays braucht auch Ihre Zeit. Erst danach kommen die zu untersuchenden Zuweisungen von value1 value2, und value3.

5.2.1 Kommentierter Assemblercode

Kopieren Sie von dem erzeugten RISC-V Assemblercode nur die Instruktionen in ihr Portfolio, die den Zeilen 14,15 und 16 des Quellcodes entsprechen, und kommentieren Sie sie, analog dem folgenden Beispiel.

```
174:        00a7d683                lhu        a3,10(a5)  # Load Half Unsigned (16 Bit) a3=*(a5+10)
```

5.2.2 Analyse des Assemblercodes

Erläutern Sie im Portfolio den Unterschied der drei Zuweisungen in dem was Sie beobachten und nennen eine mögliche Erklärung (Hinweis: Stichwort *data structure alignement*):

[6] Zur Anzeige und dem selbst Zusammenstellen von Speicherinhalten: Die Werte werden zusammengestellt aus dem Speicherinhalt von mem_ram_b3[x] bis mem_ram_b0[x] (Instanz neorv32_int_dmem_inst). Dass sind 4 einzelne Bytes mit der selben Adresse, die in GTKWave durch *Edit* → *Combine Down* (b3 steht dabei oben, b0 unten) zu einem 32 Bit Wert vereint werden können

6 Hardware Erweiterung

Um die Erweiterung des SOPC um weitere Hardware zu verstehen, zeigt das folgende Kapitel die Integration der Hardware und Software für die LED Ansteuerung. Danach soll die Ansteuerung und Integration eines Sieben-Segment Dekoders selbst umgesetzt werden.

6.1 LED Ansteuerung

Die Erweiterung um die LED Ansteuerung wird in den folgenden beiden Kapiteln nach Hardware und Software getrennt erläutert.

6.1.1 Hardware Erweiterung

Unter osflow/boards/iceduino wurde dafür die VHDL Datei *iceduino_led.vhd* hinzugefügt. CPU Peripherie wird über den Wishbone-Bus angesprochen.[7] Ob das hinzugefügte Modul angesprochen wird (lesen oder schreiben), wird über die Adresse und die Handshake Signale `cyc_i` und `stb_i` erkannt:

```
module_active <= '1' when ((adr_i = led_addr) and (cyc_i = '1' and stb_i = '1')) else '0';
```

Die entsprechende Antwort des Moduls **kann** bedingungslos im nächsten Takt wie in 5 erfolgen.

Listing 5: Handhsake Signale bedingungslos erzeugen

```
w_access: process(clk_i)
begin
    if rising_edge(clk_i) then
        -- handshake
        err_o <= '0';
        if (module_active = '1') then
            ack_o <= '1';
        else
            ack_o <= '0';
        end if;
        [...]
    end if;
end process w_access;
```

Die Auswertung auf Lesen und Schreiben auf Register erfolgt über das `we_i` Signal:

Listing 6: Simples Lesen/Schreiben

```
    --write
    if (module_active = '1' and we_i = '1') then
        reg_led <= dat_i(7 downto 0);
    end if;
    --read
    dat_o(31 downto 0) <= (others => '0');
    if (module_active = '1' and we_i = '0') then
        dat_o(7 downto 0) <= reg_led;
    end if;
```

Damit das LED-Modul mitkompiliert wird, wird es im nächsten Schritt im Makefile hinzugefügt (Siehe dort Zeile 63) und im top-level VHDL (`neorv32_iceduino_top.vhd`) instanziiert (iceduino_led_inst):

Listing 7: Instanziierung des LED Moduls

```

iceduino_led_inst: entity iceduino.iceduino_led
generic map (
    led_addr => x"F0000000"
  )
  port map (
        clk_i  => clk_50mhz,
```

[7] SoC Interconnection: WISHBONE (`https://opencores.org/howto/wishbone`)

```
        rstn_i => external_rstn,
        adr_i  => master_bus.adr_o,
        dat_i  => master_bus.dat_o,
        dat_o  => led_resp.rdata_i,
        we_i   => master_bus.we_o,
        stb_i  => master_bus.stb_o,
        cyc_i  => master_bus.cyc_o,
        ack_o  => led_resp.ack_i,
        err_o  => led_resp.err_i,
        led_o  => led
);
```

Alle mit „master_bus“ beginnenden Signale sind für alle Module gleich. Für jedes Modul individuell wird das *led_resp* Record angelegt, das aus drei Elementen besteht:

Listing 8: Slave Resp Record für Daten, Acknowledge und Error

```
-- bus_wishbone slave response  --
type slave_resp_t is record
    rdata_i : std_ulogic_vector(31 downto 0);
    ack_i   : std_ulogic;
    err_i   : std_ulogic;
end record;
constant slave_resp_default : slave_resp_t := (rdata_i => (others => '0'), ack_i => '0', err_i => '0');
signal active_slave_resp    : slave_resp_t := slave_resp_default;
signal led_resp             : slave_resp_t := slave_resp_default;
...
```

Die individuell erzeugten Signale liegen einerseits auf der led-Instanz, auf der anderen Seite werden sie von einem Bus-Multiplexer verteilt, der die Basisadresse des angesprochenen Moduls auswertet:

Listing 9: Hinzufügen der Response Signale zum Multiplexer

```
-- external bus multiplexer --
bus_multiplexer: process(master_bus, led_resp,switch_resp,button_resp)
begin
    active_slave_resp.rdata_i <= (others => '0');
    active_slave_resp.ack_i <= '0';
    active_slave_resp.err_i <= '0';
    if(master_bus.adr_o(31 downto 8) = x"F00000") then
        case master_bus.adr_o(7 downto 0) is
            when x"00" =>
                active_slave_resp.rdata_i <= led_resp.rdata_i;
                active_slave_resp.ack_i <= led_resp.ack_i;
                active_slave_resp.err_i <= led_resp.err_i;
            when x"08" =>
                active_slave_resp.rdata_i <= switch_resp.rdata_i;
                active_slave_resp.ack_i <= switch_resp.ack_i;
                active_slave_resp.err_i <= switch_resp.err_i;
            when x"10" =>
...
```

Achtung: Die Basisadresse eines neuen Moduls wird an **zwei** Stellen im Sourcecode festgelegt: 1. Bei der Instanz als generischer Wert für die interne Auswertung 2. Im Multiplexer, der die Rückgabesignale des entsprechenden wishbone-Slaves auswertet.

6.1.2 software

Für das Modul der LED wird die Headerdatei sw/lib/include/iceduino_led.h mit den Funktionsprototypen für den Zugriff auf das VHDL-Modul erstellt (z.B. `iceduino_led_set`). Unter sw/lib/source/iceduino_led.c werden die Funktionen für den Zugriff auf das Modul umgesetzt, wie in Listing 11 skizziert.

Die zentrale include-Datei für die Hardware-Peripherie `neorv32_iceduino.h` muss noch um die neue Header-Datei erweitert werden.

Listing 10: Erweiterung von neorv32_iceduino.h

```
//#io devices#
#include "iceduino_led.h"
```

```
#include "iceduino_switch.h"
#include "iceduino_button.h"
#include "iceduino_pmod.h"
#include "iceduino_gpio.h"
#include "iceduino_uart.h"
#include "iceduino_spi.h"
#include "iceduino_i2c.h"
#include "iceduino_adc.h"

//#memory mapped io#
//LED base address
#define ICEDUINO_LED_BASE (0xF0000000UL)
//LED output port 8-bit (r/w) */
#define ICEDUINO_LED (*((volatile uint8_t*) (ICEDUINO_LED_BASE)))

//SWITCH base address
#define ICEDUINO_SWITCH_BASE (0xF0000008UL)
//SWITCH input port 8-bit (r) */
#define ICEDUINO_SWITCH (*((const volatile uint8_t*) (ICEDUINO_SWITCH_BASE)))
```

Die Basis Adresse (0xF000 0000) der neuen Hardware wird hier zentral definiert und ein Pointer zum Zugriff (ICEDUINO_LED) angelegt. Das Schlüsselwort `volatile` ist beim Zugriff auf memory-mapped Hardware notwendig, um eine Optimierung des Compilers bei Speicherzugriffen zu vermeiden. Wird beispielsweise zweimal hintereinander von der Hardware gelesen, können und sollen unterschiedliche Werte gelesen werden. Denkbar ist dies beim Lesen einer seriellen Schnittstelle oder eines Analog-Digital Wandlers. Ohne `volatile` Qualifizierer wird der Compiler einen einmal gelesenen Wert möglicherweise in einem Register zwischenspeichern und ein zweites Lesen von der Adresse nicht umsetzen, da regulärer Speicher in der Regel seinen Inhalt zwischen zwei Lese-Operationen *nicht* ändert.

Listing 11: Exemplarische Hardware-Zugriffsfunktionen aus iceduino_led.c

```
#include "neorv32_iceduino.h"

//set led port (0-7)
void iceduino_led_set(int value) {
    ICEDUINO_LED = (uint8_t) value;
}

//clear output
void iceduino_led_clr() {
    ICEDUINO_LED = 0x00000000;
}
...
```

Nun kann ein Testprogramm erzeugt werden und das Verhalten mit der Simulation analysiert werden.

6.2 Aufgabe 7-Segment Dekoder

- Entwerfen Sie in VHDL einen Sieben-Segment Dekoder, der aus 4 Bit Daten die entsprechenden Segmente einer LED Anzeige ansteuert. Das sollte eine rein kombinatorischer Lösung (keine Register, kein Taktflanke) sein (z.B. bedingte Signalzuweisung)
- Fügen Sie eine SOPC Hardware an einer freien Adresse hinzu, um über das Interface 4 Bit abzulegen, die dann als 7 Bit Wert über I/Os ausgegeben werden können.
- Erstellen Sie Bibliothek und Testsoftware, um das Modul zu testen.
- Simulieren Sie ein Bare-Metal Testprogram, das durchzählt (ohne Delays, die brauchen Sie nur für die reale Umsetzung)
- Nach erfolgreichem Debuggen planen Sie die reale Umsetzung: Welche Pins können Sie für den Anschluss nutzen oder freigeben?
- Die benutzten LED Anzeigen sind vom Typ Kingbright SC03-12EWA. Entwerfen und Verdrahten Sie anhand des Datenblatts den Anschluss an das Display und vergessen Sie die Vorwiderstände nicht.
- Ergänzen Sie das Portfolio um den Code des Dekoders und der neuen Hardware, der Software, einem Screenshot der Simulation. Ergänzen Sie ein Foto des Aufbaus (Dateigröße möglichst unter 500 kb)

7 WS2812 Display

Im Rahmen der Veranstaltung soll ein benutzerdefiniertes Hardware Modul dem NEORV32-Prozessor in VHDL hinzugefügt und auf dem Iceduino Board ausgeführt werden. Mit dem zu entwerfende Hardware Modul soll es möglich sein, eine 8x8 RGB-LED Matrix als Display zu betreiben.

Die Umsetzung erfolgt in mehreren kleinen Schritten:

- Aufgabe: Aufstellen der Formeln und Entwickeln des aserial Moduls
- Aufgabe: Vom ROM auf die WS2812 Matrix schreiben
- Aufgabe: Top-Level-Entity für DPRAM und CPU Integration

Ziel ist es Game Of Life `https://de.wikipedia.org/wiki/Conways_Spiel_des_Lebens` als verteiltes System umzusetzen[8].

Über die Software soll in einer späteren Umsetzung die Möglichkeit bestehen, das 8x8 Spielfeld durch Koppeln mit anderen Boards um beliebig viele Module zu erweitern, sodass ein größeres Spielfeld entsteht.

7.1 WS2812 Ansteuerung

Im folgenden soll der Treiber für die LED-Matrix entworfen werden. Auf der LED-Matrix befinden sich 64 WS2812 RGB LEDs mit integriertem Controller. Zuverlässigere Informationen als im Datenblatt über die Ansteuerung der WS2812 LEDs finden sich hier: `https://www.mikrocontroller.net/articles/WS2812_Ansteuerung` und hier: Understanding the WS2812 Part I Die Ansteuerung der WS2812 erfolgt über eine einzelne Datenleitung mit einem asynchronen seriellen Protokoll. Eine '0' wird dabei über einen kurzen, eine '1' über einen langen High-Puls definiert. Jede LED benötigt 24 Datenbits (G8:R8:B8)[9]. Die Daten aller LEDs werden seriell direkt hintereinander übertragen. Das höchstwertigste Bit wird zuerst übertragen (MSB first, Most Significant Bit first).

Wenn die Datenleitung für mehr als 50 µs auf „low“ gehalten wird („reset code“), werden die Daten in die PWM-Register der LEDs übernommen.

Timing:
Senden einer '1' => 900 ns(±150 ns) HIGH gefolgt von 350 ns(±150 ns) LOW.
Senden einer '0' => 350 ns(±150 ns) HIGH gefolgt von 900 ns(±150 ns) LOW.

Insgesamt dauert das Senden eines Bits also 1250 ns. Die 64 RGB-LEDs sind, wie in Abbildung 5 zu sehen, in einer Kette miteinander verbunden. Die erste LED der Kette befindet sich oben Links, dort wo auf der Rückseite der LED-Matrix die Pins für 5V, DIN und GND sind. Insgesamt müssen über die Datenleitung DIN 64 x 3 x 8 = 1536 Bits hintereinander gesendet werden.

7.1.1 Aufgabe: Aufstellen der Formeln und Entwickeln des aserial Moduls

Wir arbeiten mit dem 50 MHz Takt, der am Pin 52 des FPGA anliegt. Berechnen Sie nach wie vielen Taktzyklen die Zeiten 350 ns(±150 ns) LOW, 900 ns(±150 ns) HIGH und 1250 ns Periode erreicht sind. Notieren Sie die verwendeten Formeln (Latex Math Mode) und Ergebnisse in Ihrem Portfolio

$$z.B.: \quad 6 \cdot 9 = 42$$

Entwickeln Sie ein Modul mit der folgenden Entity:

```
entity WS2812_aserial is
port (
reset       : in  std_ulogic;
clk         : in  std_ulogic;
wnr         : in  std_ulogic;
```

[8] s. dazu auch `https://youtu.be/6avJHaC3C2U?feature=shared&t=263`
[9] GRB: Grün, Rot, Blau: diese unkonventionelle Reihenfolge wird vom Protokoll vorgeschrieben

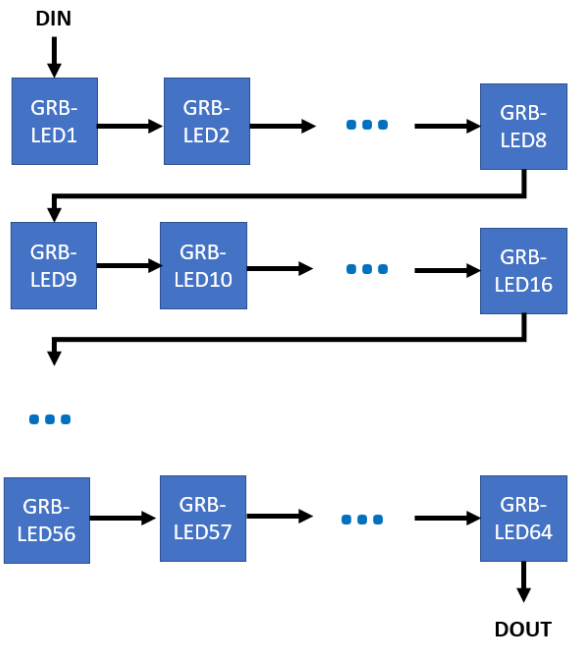


Abbildung 5: Daisy Chain der Datenleitungen der LEDs auf der Matrix-Platine

```
data_in     : in  std_ulogic;      -- Das zu sendende Bit
ws2812_out  : out std_ulogic;      -- Serieller Ausgang WS2812
run         : out std_ulogic       -- '1' => können keine neuen Daten
--          angenommen werden
);
end WS2812_aserial;
```

Es soll sich so Verhalten: wenn `wnr` = '1' ist, wird das an `data_in` anliegende Bit in der oben definierten Sequenz an den Ausgang `ws2812_out` gelegt. Der Ausgang `run` signalisiert während des Sendens der Sequenz für ein Bit mit einer '1', dass aktiv gesendet wird und wechselt anschließend auf '0'. Der Wechsel zu '0' signalisiert einem übergeordnetem Prozess (bzw. Zusatndsmaschine), dass das nächste zu sendende Bit an `data_in` angelegt werden kann. Sehr wichtig ist, dass zwischen dem Senden zweier Bits durch das Handshaking über `wnr` und `run` keine zusätzliche Pause entsteht. Da das Senden einer '1' und einer '0' bis zum Zeitpunkt 350 ns selbes Verhalten zeigen (`ws2812_out` ist high), kann das Senden des nächsten Bits bereits gestartet werden, und der übergeordneten Prozess kann das `data_in` Bit später ändern.

Entwickeln Sie ein Modul, dass das hier beschriebene Verhalten zeigt und testen Sie es mit einer einfachen VHDL-Testbench. Fügen Sie hier vier Screenshots der Simulation vom Versenden der Bit-Sequenz "1010" zu und setzen dort Marker [10], um die folgenden Zeiten zu markieren und zu messen:

1. Senden einer '1' => 900 ns(±150 ns) HIGH
2. Senden einer '1' => 350 ns(±150 ns) LOW
3. Senden einer '0' => 350 ns(±150 ns) HIGH
4. Senden einer '0' => 900 ns(±150 ns) LOW

Sehen Sie zu, dass alle Ports der Entität über den gesamten zeitlichen Verlauf der vier gesendeten Bits mit aufgenommen werden, um die Funktion des o.g. Handshakings beurteilen zu können. Übernehmen Sie den aserial Code, Testbench Code und die Screenshots in das Portfolio.

7.2 Aus einem Speicher die WS2812 Matrix beschreiben

Hier setzen Sie die Komponente RAMtoWS2812 um. In einem ersten Test ersetzen Sie das Dual-Port RAM mit einem vorgegebenem ROM, um mit diesem einfacheren Setup die Darstellung auf der 8x8

[10] In *GTKWave* einstellen der Zeitbasis auf Nanosekunden: Menü View → Scale to Time Dimension → ns. Setzen der Marker mit linker und mittlerer Maustaste.

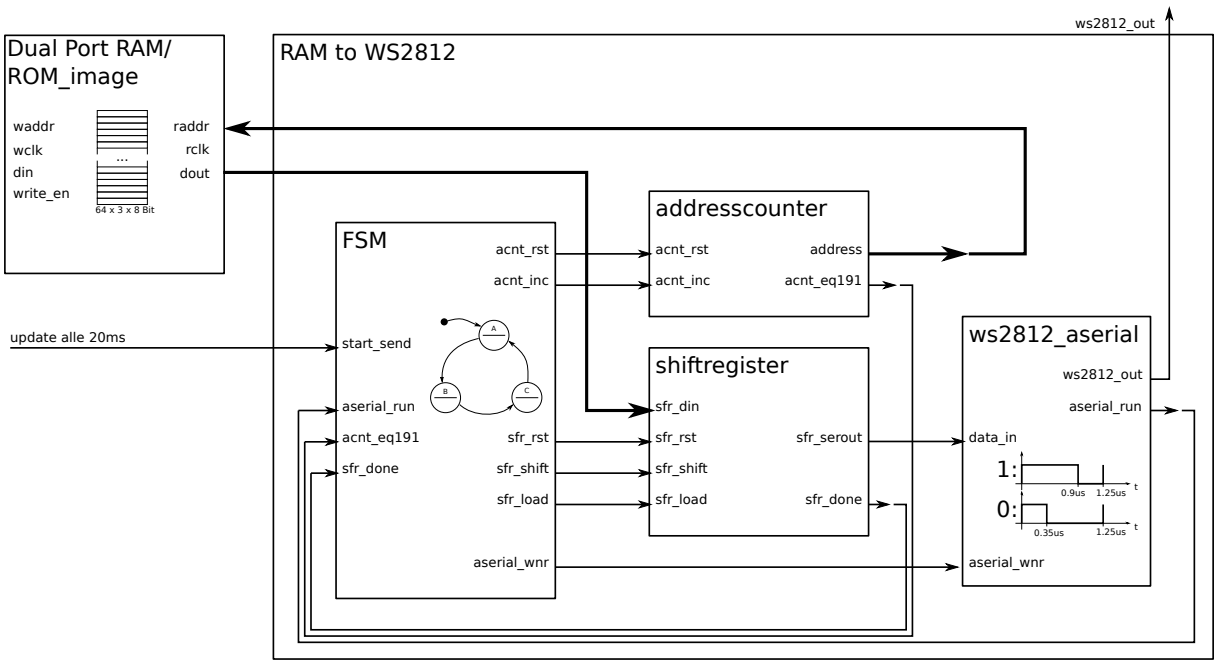


Abbildung 6: Vorschlag Blockschaltbild (Prozesse oder Komponenten) der Ansteuerung der WS2812 LED Matrix

LED Matrix zu testen.

Abbildung 6 zeigt eine mögliche Umsetzung, wie die im RAM vorliegenden Daten an die WS2812 Matrix versendet werden können. Das RAM ist als 8-Bit RAM organisiert, d.h., dass an jeder Adresse, die über *raddr* adressiert werden kann ein 8 Bit Wert über *dout* ausgelesen werden kann. Für die 64 LEDs der Matrix werden je LED 3 x 8-Bit Werte benötigt, so dass der Speicher mindestens 192 x 8-Bit Werte adressieren sollte. Da $log_2(192) \approx 7,5$ sind mindestens 8 Adressleitungen notwendig, um alle Werte adressieren zu können. Das Zeitverhalten des RAMs: Einen Takt nach Ändern der Adresse kann der Wert der Speicherstelle ausgelesene werden.

Die vorgeschlagene Umsetzung der *RAMtoWS2812*-Komponente besteht aus vier Prozessen und/oder Komponenten:

1. **FSM**: (Finite State Machine = endliche Zustandsmaschine[11]) als Ablaufsteuerung der einzelnen Komponenten. Kann mit einem oder mehreren Prozessen oder als eigenständige Komponente umgesetzt werden.
2. **addresscounter**: Ein Zähler, der die 192 Adressen des Speichers hochzählt und als 8-Bit Wert auf dem Ausgang *address* ausgibt.
   - über *acnt_rst* setzt die FSM den Zähler auf 0 zurück.
   - mit *acnt_inc* aktiviert die FSM den Zähler. Wird dieser Eingang für einen Takt auf high gelegt, wird der Zähler auch nur um Einen Zählwert hochgezählt.
   - *acnt_eq191* wird aktiv, wenn der Zählerstand 191 beträgt und signalisiert der FSM, dass der Durchlauf beendet ist.
3. **shiftregister**: Das Schieberegister wird als Serialisierer benutzt. Es kann einen 8-Bit Wert in einem Zyklus laden und gibt davon den Wert des MSB am Ausgang *sfr_serout* aus.
   - Über *sfr_rst* wird das Register und der intene Zähler zurückgesetzt.
   - mit einem aktiven *sfr_shift*-Signal erreicht die FSM, dass das Register um eine Stelle nach links geschoben wird, so z.B. der Wert an Stelle 6 auf Stelle 7 rutscht. Gleichzeitig wird intern mitgezählt, wie oft bereits geschoben wurde.
   - mit *sfr_load* wird das Schieberegister in einem Taktzyklus mit dem 8-Bit Wert geladen, der an *sfr_din* anliegt.

[11] i.d.R. dargestellt als Zustandsüberführungsdiagramm nach Moore oder Mealy.

- *sfr_done* zeigt der FSM an, dass bereits 7 Mal geschoben wurde und somit das letzte Bit (LSB) der 8 Bit am Ausgang anliegt. Die FSM wird dann die nächste Adresse laden und das Schieberegister zurücksetzen und mit dem neuen 8-Bit Wert laden.

4. **ws2812_aserial**: die im Kapitel WS2812 Ansteuerung entwickelte Komponente zur Erzeugung des seriellen WS2812 Protokolls für ein Bit wird hier eingesetzt. Der erzeugte serielle Bitdatenstrom wird über den Port *ws2812_out* an die höhere Instanz und den Ausgangsport zur LED-Matrix weitergegeben.

   - Die FSM aktiviert die Versendung eines Bits über *aserial_wnr*. Wie im Kapitel WS2812 Ansteuerung beschrieben soll das Handshaking beachtet werden. *aserial_wnr* soll durchgehend aktiv bleiben und erst beim Versenden des letzten Bits deaktiviert werden. *serial_run* zeigt der FSM an, dass ein Bit aktiv versendet wird, wird aber mindestens für einen Takt deaktiviert, wenn das nächste Bit versendet wird.

7.2.1 Aufgabe: Vom ROM auf die WS2812 Matrix schreiben

Entwerfen Sie RAMtoWS2812. Die bereitgestellten ROM-Images finden sich komplett hier im git des Scripts, in Listing 12 sind die ersten Zeilen des Packages enthalten. Das VHDL-Package enthält als ROM mehrere konstante Arrays (rom_image1 bis rom_image8), die unterschiedliche Bilder darstellen. Das Einbinden des Packages und die Nutzung der Konstanten zeigt Listing 13.

Listing 12: Erste Zeilen des packages *romimages.vhd*

```
library ieee;
use ieee.std_logic_1164.all;

--  8 Bit rom_image (192 Adressen Typ grb_bmp_t)  : rom_image1   bis rom_image7
-- 32 Bit rom_image ( 64 Adressen Typ grb_bmp32_t): rom32_image1 bis rom32_image7

package const_image is
 type grb_bmp_t is array (0 to 191) of std_ulogic_vector(7 downto 0);
 type grb_bmp32_t is array (0 to 63) of std_ulogic_vector(31 downto 0);

constant rom_image1 : grb_bmp_t := (
    000 => x"00",
    001 => x"00",
    002 => x"00",
```

Listing 13: Nutzen der in *romimages.vhd* definierten konstanten Arrays als ROMs

```
        library ieee;
        use ieee.std_logic_1164.all;
        use ieee.numeric_std.all;    -- für die Konvertierungsfunktionen

        library work;
        use work.const_image.all;    -- Einbinden der romimage Arrays
        ...
        signal raddr  : std_ulogic_vector(7 downto 0);  -- Statt Adress-Ausgang an DP-RAM
        signal dout   : std_ulogic_vector(7 downto 0);  -- Statt Daten-Eingang von DP-RAM
        ...
        dout <= rom_image1(to_integer(unsigned(raddr)));  -- Adressdemultiplexer für ROM
```

1. Entwerfen und zeichnen Sie das Zustandsüberführungsdiagramm der FSM und fügen Sie es im Portfolio ein.

2. Simulieren Sie den Durchlauf und erstellen Screenshots für das Portfolio, um alle Übergänge Ihres Zustandsautomaten zeigen. Vergessen Sie nicht die Zustandsübergänge bei Erreichen der letzten Adresse.

3. Synthetisieren Sie ihr Design und Testen es mit einem ROM-Image und der bereitgestellten Hardware zunächst ohne NEORV32 CPU. Versorgen Sie die WS2812-Matrix mit 5V und GND von den Arduino-Headern des Iceduino. Benutzen Sie als WS2812 Ausgang jetzt bereits den Ausgangspin 3 des PMOD3 Headers, P1_3 (s. PMOD Header), der mit Pin 2 des FPGA verbunden ist. Dann kann der Iceduino mit Ihrem Design nach Abschluss der Entwicklung auf das Multi-Iceduino Baseboard ohne Umkonfigurieren aufgesteckt werden, s. Anhang C.

Dokumentieren Sie im Portfolio mit einem Bild des Aufbaus (Anzeige einer der bereitgestellten Images) die Funktion Ihres Entwurfs. Der hier entstandene Code (RAMtoWS2812 incl. möglicher submodule) sollte ebenfalls im Portfolio abgelegt sein, z.B. in einem gesonderten Anhang.

7.3 Dual-Port-RAM

Auf einem FPGA befindet sich typischerweise ein Block Memory, welcher als Block RAM (BRAM) bezeichnet wird. Dieser kann zum Speichern von großen Datenmengen in einem FPGA verwendet werden. Ein solcher BRAM befindet sich als integrierte Komponente auf dem ICE40HX4K FPGA. Er kann unter anderem als Dual-Port-RAM (DPRAM) verwendet werden, sodass gleichzeitig ein Prozess in den RAM schreiben und ein Prozess aus dem RAM lesen kann. Ein DPRAM zeichnet sich dadurch aus, dass es zwei verschiedene Adressierungsmöglichkeiten gibt. Hier genannt waddr (für write-address) und raddr (für read-address).

Um den integrierten Blockspeicher des FPGAs zu nutzen, müssen Sie eine entsprechende Komponente des Dual-Port-RAM anlegen und in Ihr Design implementieren. Das Synthese Tool, welches alle nutzbaren Komponenten des FPGAs kennt, erkennt automatisch anhand der VHDL-Prozesse (s. Listing 14) im Dual-Port-RAM, dass eine solche Komponente beschrieben wird und konfiguriert diese entsprechend als Dual-Port-RAM.

Listing 14: VHDL Dual-Port-RAM

```
library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;
entity ram is
    generic (
         addr_width : natural := 8;--512x8
         data_width : natural := 8);
    port (
         write_en : in std_ulogic;
         waddr : in std_ulogic_vector (addr_width - 1 downto 0);
         wclk : in std_ulogic;
         raddr : in std_ulogic_vector (addr_width - 1 downto 0);
         rclk : in std_ulogic;
         din : in std_ulogic_vector (data_width - 1 downto 0);
         dout : out std_ulogic_vector (data_width - 1 downto 0));
    end ram;
    architecture rtl of ram is
        type mem_type is array ((2** addr_width) - 1 downto 0) of
                std_ulogic_vector(data_width - 1 downto 0);
        signal mem : mem_type;
    begin
        process (wclk)
        -- Write memory.
        begin
            if (wclk'event and wclk = '1') then
                if (write_en = '1') then
                    mem(to_integer(unsigned(waddr))) <= din;
                -- Using write address bus.
                end if;
            end if;
        end process;


     -- Read memory.
     process (rclk)
     begin
        if (rclk'event and rclk = '1') then
            dout <= mem(to_integer(unsigned(raddr)));
            -- Using read address bus.
        end if;
     end process;
end rtl;
```

Der Dual-Port-RAM des Boards soll so genutzt werden, dass über den wishbone Bus über eine Adresse ein 32-Bit Wert mit den RGB-Werten in den write-port des DPRAM geschrieben werden. Auf der read-Seite werden dann die Werte gelesen.

Besonderes Augenmerk ist auf die unterschiedlichen Datenbus-Breiten zu legen (CPU: 32 Bit, WS2812 Logik: 8 Bit)

7.3.1 Aufgabe: Top-Level-Entity für DPRAM und CPU Integration

Entwerfen Sie die top level Entity `neorv_ws2812`, welche die bisher entworfenen Module implementiert und eine Schnittstelle zur CPU und zum WS2812 Pin darstellt.

Instanziieren Sie hier das in Listing 14 beschriebene Modul.

Der Dual-Port-RAM wird mit den folgenden Werte generisch konfiguriert:

```
entity ram is
generic (
addr_width : natural := 8;--512x8
data_width : natural := 8);
port (
...
```

Sie wollen für die 8x8 Matrix insgesamt 64x24 = 1.536 Bits speichern. Ist der RAM hierfür korrekt konfiguriert? Verbinden Sie die Instanz des im Kapitel 7.2 entworfenen Moduls (Auslesen vom ROM, Ansteuern der RGB LEDs) mit den read-Signalen der Instanz des Dual-Port-RAM. Die FSM kann durchlaufen (`start_send` konstant auf 1) und muss nicht extra alle 20 ms getriggert werden.

Entwerfen Sie die Anbindung an den write Port des DPRAMS über den wishbone Bus. Für jedes der 64 Pixel sollte eine 32-Bit Adressen reserviert sein, von der nur 3x8 Bit für RGB genutzt werden.

- Bits 31..24 : reserviert
- Bits 23..16 : Rot
- Bits 15..8 : Grün
- Bits 7..0 : Blau

Schreibt die CPU auf eine der Pixel Adressen, sollte der DPRAM entsprechend beschrieben werden. Hier gibt es große Freiheitsgrade, die die Umsetzung vereinfachen, z.B. durch Änderung der Speicherarchitektur (64 x 32 Bit statt 192 x 8)

- Testen Sie das Schreiben ins DPRAM über eine Testsoftware und dokumentieren im Portfolio, dass die von der CPU geschriebenen Werte in der WS2812-Logik ankommen
- Testen Sie das Schreiben ins DPRAM an der Hardware und dokumentieren die Funktion über ein Foto im Portfolio
- Fügen Sie die Hardware-Erweiterungen und die Testsoftware ins Portfolio ein (z.B. in einen separaten Anhang)

8 Game Of Life

Im Rahmen der Veranstaltung wird zunächst ein lokales Game of Life auf dem iceduino Board umgesetzt, das auf einer 8x8 RGB-LED Matrix betrieben wird. Dafür werden die Game- of Life Regeln als C-Programm umgesetzt und auf der 8x8 Matrix angezeigt. Sehen Sie vor, dass die Farbe für lebende und tote Zellen konfigurierbar bleibt.

Eine gute Beschreibung des Game of Life finden Sie in Wikipedia:
`https://de.wikipedia.org/wiki/Conways_Spiel_des_Lebens`

In einem weiteren Schritt werden die einzelnen 8x8 Module der beteiligten Projektgruppen zu einem großen Modul zusammengesetzt und tauschen die Informationen über ihre Nachbarzellen über eine serielles Protokoll aus. Die Steuerung wird dann über eine zentrale Software umgesetzt, die das initiale Muster auf alle beteiligten iceduinos verteilt, den Austausch der Informationen über die Nachbarzellen am Rand organisiert und dann per externen Signal die Berechnung der nächsten Generation bei allen beteiligten Boards gleichzeitig auslöst.

8.1 GoL Software

Nutzen Sie den Bootloader für ihre Software Entwicklung, so können Sie leicht neue Software Versionen hochladen ohne die langwierige Neu-Synthese der FPGA Konfiguration.

Für die Datenhaltung des Game of Life Feldes mit allen zu berücksichtigen Zellen siehe Abbildung 7. Die blauen Felder entsprechen den sichtbaren 64 Game of Life Feldern. Die orangen Felder stehen für die unsichtbaren Nachbarn, die für die Berechnung der nächsten Generation der Felder am Rand benötigt werden. Die Pfeile zeigen an, welche Nachbarfelder den Wert der nächsten Generation beeinflussen. Somit zeigen immer acht Pfeile auf jedes sichtbare Feld. Z.B. benötigt das Feld L(7)(7) zur Berechnung seiner nächsten Generation die unsichtbaren Felder N(17), N(18) und N(19). Im standalone Betrieb können diese Felder als 0, also nicht lebend angenommen werden,- im späteren Gemeinschaftsbetrieb müssen diese Felder mit den Inhalten der Nachbarn geladen werden.

Das SoPC im iceduino hat begrenzten RAM Speicher, es ist also ratsam Felder, die lediglich als lebendig (1) oder tot (0) dargestellt werden, als Bits abzuspeichern. Es gibt 64 sichtbare Bits (=8 Byte) und 36 unsichtbare Bits (4x8 an den Seiten + 4 Ecken), also 100 Bits insgesamt.

Dies ist insbesondere zu empfehlen, da für die Berechnung der Folge-Generation ein zweites Feld genutzt werden sollte, um die Ergebnisse (Anwendung der GoL Regeln) abzuspeichern, ein Ändern im selben Feld führt zu Inkonsistenzen bei der Berechnung der Folgegeneration, da es hier zur Vermischung von Feld-Zuständen der nächsten und aktuellen Generation kommt.

Das Auslösen der Berechnung und Anzeige der nächsten Generation soll über eine negative Taktflanke an einem Taster erfolgen (aktiv low). Das Einlesen der Taster erfolgt über Software und es sollte ein Entprellen vorgesehen werden, das z.B. durch eine Warteschleife (delay) von 150 ms ***nach*** der Darstellung der neuen Generation erreicht wird.

Um die Kommunikation über das MMCP Protokoll vorzubereiten, sollte jetzt bereits daran gedacht werden, dass der Datenaustausch in 8 Byte Blöcken (im folgenden L7_SDU[0..7]) erfolgt. Der folgende Unterabschnitt gibt einen Ausblick auf das *danach* zu implementierende Kommunikationsprotokoll.

8.1.1 Kommunikationsprotokoll

ApNr 1: set Muster

Der Inhalt der ankommenden SDU von **ApNr 1** stellt das Initial Muster des GoL-Muster in der folgenden Form dar:

ApNr 1:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
GoL-Bit:	63 .. 56	55 .. 48	47 .. 40	39 .. 32	31 .. 24	23 .. 16	15 .. 8	7 .. 0

Initialisieren Sie Ihre GoL-Matrix mit den empfangenen Werten. Schauen Sie sich hierfür Abbildung 7 an. Dabei ist die Matrixposition L(x)(y) = GoL-Bit(n) (mit x=n%8 und y=n/8).

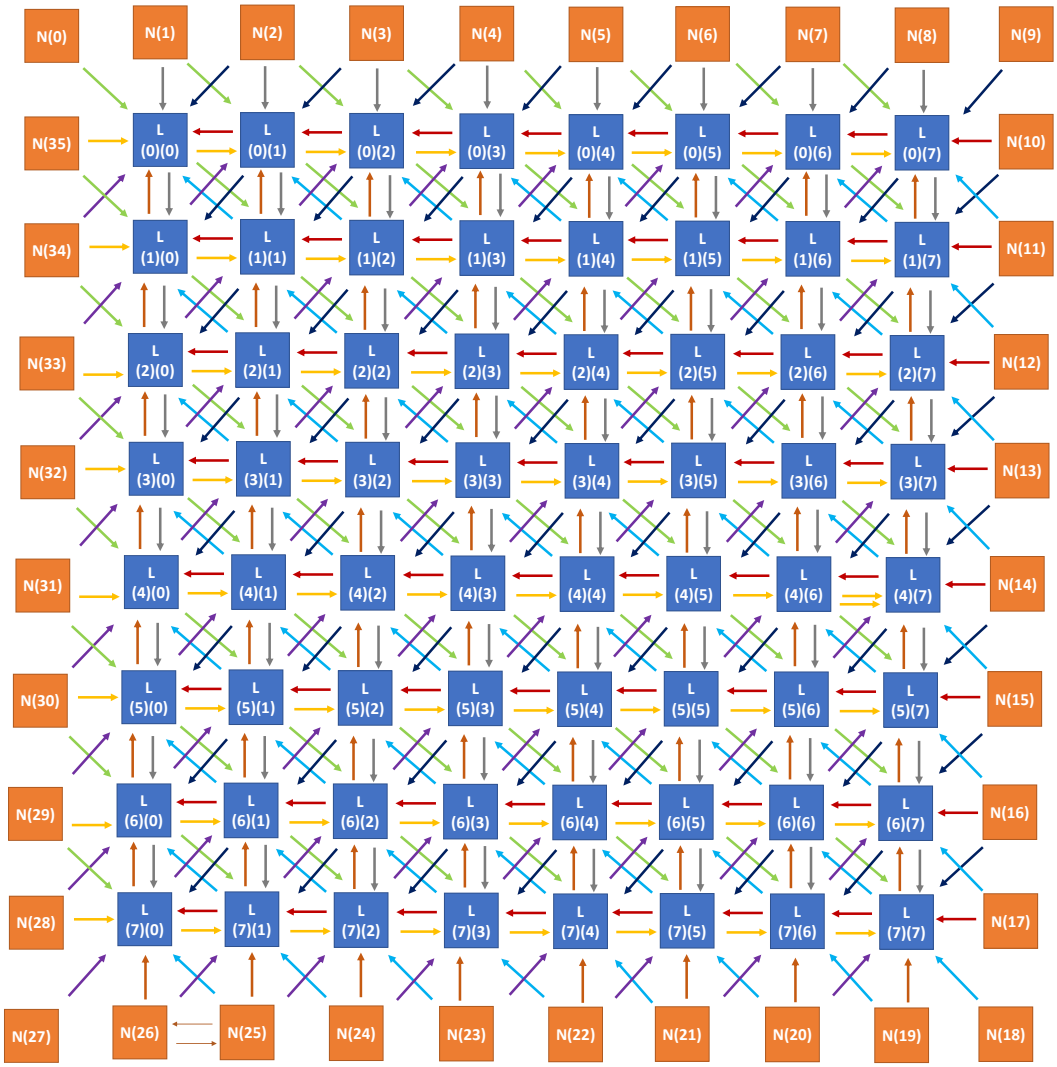


Abbildung 7: Indizes der `inputs`-Matrix der 8 Bit breiten N-Eingänge

Sende Sie die empfangene Nachricht ohne die SDU zu ändern an den Master zurück.

ApNr 2: NextGen

Der Inhalt der ankommenden SDU von **ApNr 2** wird ignoriert. Wenn **ApNr 2** empfangenen wird, soll eine neue Generation berechnet werden. Das ist das Gleiche Verhalten wie beim Drücken des Tasters. Senden Sie die empfangene Nachricht ohne die SDU zu ändern an den Master zurück.

ApNr 3: read Life

Der Inhalt der ankommenden SDU von **ApNr 3** ist leer.
Das eigenen Muster der GoL soll an den Master zurückgesendet werden.

Füllen Sie hierzu die Zustände ihre 64 GoL-Zellen in die SDU in folgender Form ein:

ApNr 3:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
GoL Bit:	63 .. 56	55 .. 48	47 .. 40	39 .. 32	31 .. 24	23 .. 16	15 .. 8	7 .. 0

Füllen Sie die L7_SDU wie oben dargestellt mit Werten aus Ihrem Speicher. Schauen Sie sich hierfür Abbildung 7 an. Dabei ist GoL-Bit(n) = L(n/8)(n%8). Senden Sie die beschriebene SDU an den Master zurück.

ApNr 4: send color Dead

Der Inhalt der ankommenden SDU von **ApNr 4** stellt den RGB-Wert für tote Zellen in der folgenden Form dar:

ApNr 4:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
Color Bit:	R(7 .. 0)	G(7 .. 0)	B(7 .. 0)	-	-	-	-	-

Initialisieren Sie den RGB-Wert für tote Zellen entsprechend mit den empfangenen Werten und senden Sie die empfangene Nachricht ohne etwas an der L7-SDU zu ändern an den Master zurück.

ApNr 5: send color Life

Der Inhalt der ankommenden SDU von **ApNr 5** stellt den RGB-Wert für lebende Zellen in der folgenden Form dar:

ApNr 5:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
Color Bit:	R(7 .. 0)	G(7 .. 0)	B(7 .. 0)	-	-	-	-	-

Initialisieren Sie den RGB-Wert für lebende Zellen entsprechend mit den empfangenen Werten und sende Sie die empfangene Nachricht ohne etwas an der L7-SDU zu ändern an den Master zurück.

ApNr 6: send Nachbarn

Der Inhalt der ankommenden SDU von **ApNr 6** stellt die Werte der Nachbar-Zellen in der folgenden Form dar:

ApNr 1:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
N Bit:	-	-	-	’0000’ & 35 .. 32	31 .. 24	23 .. 16	15 .. 8	7 .. 0

Die N Bits 0-35 entsprechen den Nachbarn aus Abbildung 7. Initialisieren Sie beim Empfangen der Nachricht die Nachbarzellen in Ihren Speichern und senden Sie die empfangene Nachricht ohne die SDU zu ändern an den Master zurück.

8.1.2 Aufgabe

- Implementieren Sie ein einfaches Game Of Life ohne Kommunikation, welches beim Reset ein initiales Muster aus dem Speicher lädt und auf Knopfdruck die nächste Generation berechnet und darstellt.
- Stellen Sie im Portfolio dar, wie sie die Speicher organisieren und auf einzelne Zellen zugreifen, z.B. anhand von Code Beispielen.
- Stellen Sie sicher, dass der komplette C-Code im git vorhanden ist und verweisen aus dem Portfolio darauf.
- Testen Sie Ihre Implementation mit einem Oszillator, z.B. dem Oktagon (passt auf 8x8 und hat 5 Zyklen,s. wikipedia) und dokumentieren dies mit Fotos.
- Testen Sie die korrekte Auswertung der unsichtbaren Nachbarzellen: In den Abbildungen 8, 9 und 10 werden die Generationen des Pulsars auf einer 15x15 GoL-Matrix dargestellt: `https://upload.wikimedia.org/wikipedia/commons/0/07/Game_of_life_pulsar.gif`
- Dokumentieren Sie die korrekte Funktion der unsichtbaren Zellen im Portfolio mit Fotos der drei Generationen.

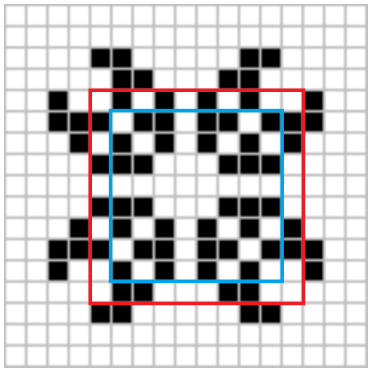


Abbildung 8: Muster 1

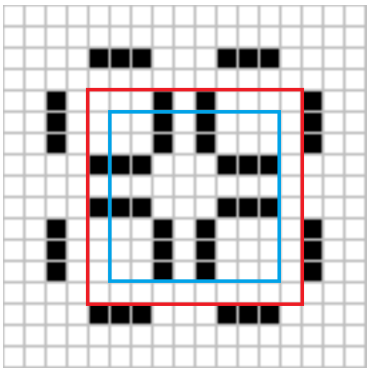


Abbildung 9: Muster 2

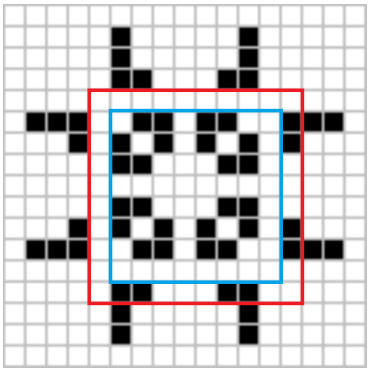


Abbildung 10: Muster 3

Das Muster des Pulsars wiederholt sich nach 3 berechneten Generationen. Das Blau umrahmte Feld soll Ihre 8x8 GoL-Matrix darstellen. Zwischen dem blauen und roten Rahmen befinden sich die unsichtbaren Felder, die direkten Einfluss auf die sichtbare GoL-Matrix haben.

9 Implementierung der Kommunikation

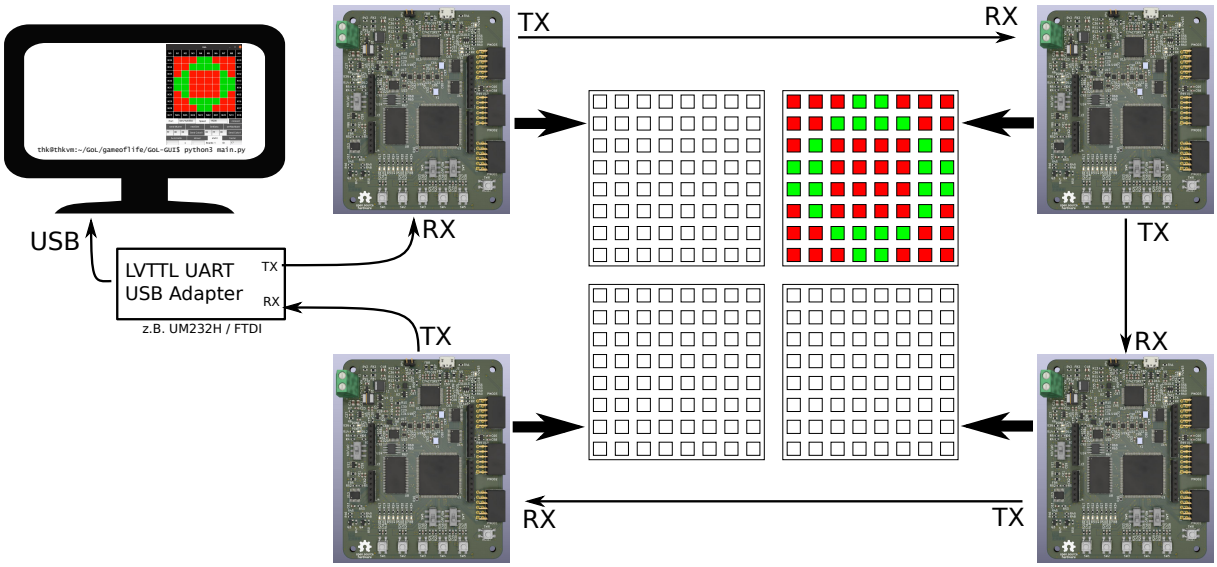


Abbildung 11: Kommunikationskette am Beispiel von vier Teilnehmern

Der nächste Schritt ist es, die Kommunikation für das GoL zu implementieren, sodass unser 8x8 Spielfeld um beliebig viele 8x8 Spielfelder auf anderen Boards erweitert werden kann. Machen Sie sich hierfür zunächst mit dem Kapitel 9.5 vertraut, um Verständnis für die Fachbegriffe und Voraussetzungen für das hier vorgestellte OSI Modell zu erlangen.

Das MMCP implementiert einige der OSI Layer und berücksichtigt die besondere Topologie der Daisy Chain. Wie in Abbildung 11 skizziert, werden alle Daten reihum geschickt. Der Master/PC schickt die Informationen los. Der erste Teilnehmer empfängt die Daten und schickt sie an den nächsten Teilnehmer, wenn er nicht adressiert wurde. Ist die Nachricht beim adressierten Teilnehmer eingetroffen, wird sie beantwortet und weitergeschickt. Alle folgenden Teilnehmer reichen sie weiter und der Master empfängt die Antwort. Kommt das Paket unbeantwortet wieder beim Master an, hat sich kein Teilnehmer angesprochen gefühlt, es war also keiner adressiert. Grundsätzlich initiiert **ausschließlich** der Master einen Datenaustausch. Ein adressierter Teilnehmer antwortet, alle anderen leiten die Pakete weiter. Teilnehmer senden nie ein Paket ab, ohne vorher eine Anforderung vom Master empfangen zu haben.

Übersicht über die Schichten:

- Anwendungsschicht: Das erste Byte bestimmt den Nutzdateninhalt, also z.B. ob die Nutzdaten das Initialmuster der GoL-Matrix darstellen oder ob die Nutzdaten mit dem aktuellen Muster der GoL-Matrix an den Master zurückgesendet werden sollen. Die folgenden 8 Byte sind für Nutzdaten vorgesehen. Wie sie gefüllt werden, bestimmt das erste Byte.
- Vermittlungsschicht: Die Daten, die gesendet werden, bekommen 4 Bytes hinzugefügt: Empfänger-Adresse, Sender-Adresse, Versionsnummer und Hops, ein Zähler, wie oft die Nachricht weitergeleitet wurde.
- Sicherungsschicht: Die übertragenen Daten bekommen ein Byte CRC-8 Prüfsumme angehängt. Beim Empfang wird die Prüfsumme ausgerechnet. Stimmt sie, werden die Daten an die höhere Schicht weitergegeben, ansonsten verworfen.
- Bitübertragungsschicht 1: Die Nutzdaten werden mit jeweils einem Byte eingerahmt, um Fehler beim Start und Ende der Übertragung zu minimieren. Der Sender sendet eine 0, der Empfänger ignoriert das erste und letzte Byte.
- Bitübertragungsschicht 2: Hier wird der in VHDL modellierte UART1 des NEORV32 Prozessors, für eine asynchrone Daten-Übertragung in der Daisy Chain genutzt.

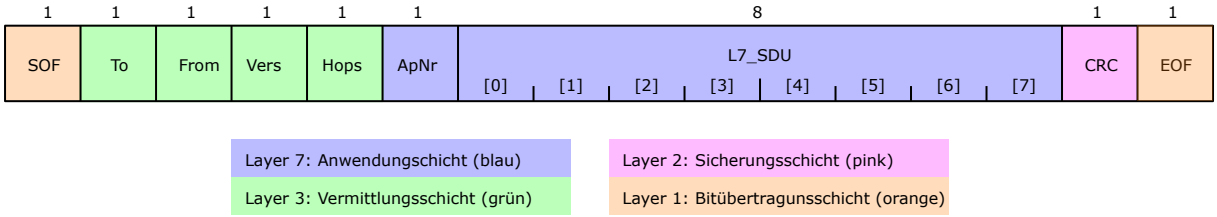


Abbildung 12: MMCP Übersicht: Protokollfelder, Schichten und Längen, jedes Kästchen: 1 Byte

9.1 Aufgabe: UART auf dem NEORV nutzen

Zuerst müssen Sie den 2. UART für den NEORV32 aktivieren. Ändern Sie hierzu die folgende Zeile in der `neorv32_iceduino_top.vhd` Datei:

```
IO_UART1_EN                    => true,  -- implement secondary UART (UART1)?
```

Dazu müssen ebenfalls in der `neorv32_iceduino_top.vhd` Datei die PINs der UART1 Komponente mit folgenden Pins des iceduino Boards verbunden werden:

- RX des UART1 wird auf den PMOD Header 1 Pin 7 (Schaltplansignal P7_1) gelegt, welcher mit dem FPGA Pin 137 verbunden ist (Schaltplan PMOD Header)
- TX des UART1 wird auf den PMOD Header 1 Pin 1 (Schaltplansignal P1_1) verbunden, welcher mit dem FPGA Pin 136 verbunden ist (Schaltplan PMOD Header)
- Eingang zur Berechnung der nächsten Generation: PMOD Header 2 Pin 1 (Schaltplansignal P1_2) = Pin 20 des FPGAs, das Signal ist aktiv low!

Die Festlegung auf diese Pins erfolgt, damit bei der Zusammenführung aller Projektteile mit dem Multi-Iceduino Baseboard (s. Anhang C) die Kommunikations-Daisy-Chain geschlossen werden kann.

```
-- secondary UART1 (available if IO_UART1_EN = true) --
uart1_txd_o     => ?, -- UART1 send data
uart1_rxd_i     => ?, -- UART1 receive data
```

Im C-Code muss der UART initialisiert werden und kann mit den im Listing 15 dargestellten Funktionen genutzt werden.

Tipp: Sie können UART0 nutzen, um Ihren Code zu Debuggen. Über UART0 läuft auch standardmäßig der Bootloader. Sollten Sie diesen in Ihrem NEORV32 noch nicht aktiviert haben, empfiehlt es sich ihn an dieser Stelle zu aktivieren, damit anschließend nur noch der C-Code compiliert und programmiert werden muss. Sehen Sie sich hierfür die Dokumentation und den User Guide des NEORV32 an. Nachdem der Bootloader beendet ist, kann der UART0 für die Debug-Ausgaben genutzt werden.

Um die Kommunikation über UART1 mit einem PC zu testen, kann ein USB/UART Wandler an den PMOD Header angeschlossen werden.

Listing 15: Initialisierung und Nutzung beider UARTs

```
1   #include <neorv32_iceduino.h>
2   // UART BAUD RATE:
3   #define BAUD_RATE 19200
4   ....
5   // Hex-Werte für Umwandlung:
6   char values[] = {'0','1', '2','3','4','5','6','7','8','9','A','B','C','D','E','F'};
7   // Speicher zum empfangen von 16 Byte:
8   uint8_t uart_receive16[16];
9   // Methode um die 16 empfangenen Bytes auszugeben:
10  uint8_t to_char_kommu(char* string, uint8_t value[]);
11  // Zeichenkette für Ausgabe:
12  char outputKommu[50];
```

```
    ...
    int main() {
        // Init UART0:
        neorv32_uart0_setup(BAUD_RATE, PARITY_NONE, FLOW_CONTROL_NONE);
        // Init UART1
        neorv32_uart1_setup(BAUD_RATE, PARITY_NONE, FLOW_CONTROL_NONE);
        // print String over UART0:
        neorv32_uart0_printf("kommunikation gestartet\n");
        ....
        while(1){
            for(int x = 0; x < 16; x++){                    //Warte auf 16 Bytes
                uart_receive16[x] = neorv32_uart1_getc();    //ein Byte empfangen, Wenn es verfügbar ist
            }
            to_char_komu(outputKommu, uart_receive16); // Byte Array in ASCII-Zeichen umwandeln
            L1_receive(uart_receive16[x]); //
            neorv32_uart0_printf("empfangen:"); // Ausgabe der 16 Byte
            neorv32_uart0_printf(outpuKomu);
        }
    }

....
uint8_t to_char_komu(char* string, uint8_t value[]){
    string[0] = '[';
    uint8_t y = 1;
    for(x = 0; x < 16; x++){
        string[y+1] = values[value[x] & 0x0F];
        value[x] = value[x] >> 4;
        string[y] = values[value[x] & 0x0F];
        string[y+2] = ':';
        y=y+3;

    }
    string[y-1] = ']';
    string[y] = '\n';

    return 0;
}
```

9.2 Aufgabe: Implementierung des Kommunikationsprotokoll

Um eine reibungslose Kommunikation zu gewährleisten brauchen Sie spätestens bei dem Zusammenschluss aller Teilnehmer eine eindeutige Adresse. Die können Sie jetzt als globalen uint_8 Wert anlegen (`uint8_t myAddress`).

In den folgenden Aufgaben sollen Sie Felder anlegen. Sie sollten die Größen der entsprechenden Felder vorher berechnen oder im Aufgabentext finden und in ihrem Code/Header anlegen. Für den Layer 7 ist das hier schon vorgegeben. Weiterhin sind im Text genutzte Variablen/Defines vorgegeben

```
/* GLOBALS */
uint8_t myAddress = 0x2a;

/* DEFINES */
#define MMCP_MASTER_ADDRESS 0
#define MMCP_VERSION 3
#define POLYNOMIAL 0x07  // CRC Polynom

#define L7_PDU_size 9
#define L7_SDU_size 8
#define L7_PCI_size 1

#define L3_PDU_size ?
#define L3_SDU_size ?
#define L3_PCI_size ?

#define L2_PDU_size ?
#define L2_SDU_size ?
#define L2_PCI_size ?

#define L1_PDU_size ?
#define L1_SDU_size ?
#define L1_PCI_size ?
```

9.3 MMCP Protokoll Layer

9.3.1 L1_receive

In der Main-loop Ihres C-Code pollen Sie permanent auf das Empfangen eines Bytes. Wenn alle 16 Bytes des Kommunikationsprotokoll empfangen wurden, wird `L1_receive(uint8_t L1_PDU[])` aufgerufen. Kopieren Sie nur die relevanten Bytes (ohne das 1. Byte SOF und das letztes Byte EOF, die PCI dieses Layers) in ein neu angelegtes Feld `uint8_t L1_SDU[])`. Die Weitergabe dieses Felds erfolgt durch Aufruf der folgenden Funktion:

9.3.2 L2_receive

Diese Funktion bekommt die SDU des L1 als PDU, deshalb ist der Prototyp: `L2_receive(uint8_t L2_PDU[])`. Berechnen und Prüfen Sie den CRC. Nur wenn die Prüfung des CRC erfolgreich ist, geben Sie die 13 Byte große L2_SDU weiter an den nächsten Layer. Stimmt die CRC Summe nicht, ist keine weitere Aktion nötig. Über einen Timeout wird der Absender erfahren, dass das Paket verlorengegangen ist. Die CRC Berechung ohne Reflektion inititalisiert das 1-Byte Register mit 0 und nutzt **0x07** als Generatorpolynom. Konkrete Hinweise zur CRC-Berechnung finden sich auf den hier verlinkten Seiten von Michael Barr. Eine online Überprüfung Ihres CRC-Codes können Sie hier durchführen.

9.3.3 L3_receive

Hier wird geprüft, ob ein Paket vom Master gesendet wurde, oder an ihn gesendet wird. Andere Adressierungsarten sind nicht zugelassen. Prototyp: `L3_receive(uint8_t L3_PDU[])`. Kopieren Sie die SDU in ein neues Feld `uint8_t L3_SDU[])`. Hier entscheidet sich, ob das Paket für Sie bestimmt ist, also an den höheren Layer 7 (4,5 und 6 sind nicht spezifiziert) weitergegeben wird, oder weitergeleitet wird, also über L2_send zurückgegeben wird. Die PCI dieses Layers hat 4 Protokollfelder:

- To: Die Empfängeradresse. Stimmt diese mit der eigenen Adresse `uint8_t myAddress` überein **und** ist der Absender 0 (also der Master), wird die 9 Byte große SDU an den Layer 7 durch Aufruf von L7_receive weitergegeben. Ist das To-Feld 0, wird die Nachricht immer weitergeleitet (Eine Ausnahme gibt es: Wenn To- und From-Feld jeweils 0 sind wird das Paket verworfen)
- From: Kommt ein Paket vom Master, so steht hier eine 0. Wenn es ein anderer Wert ist, so handelt es sich um ein Antwortpaket eines anderen Teilnehmers, das wir weiterleiten müssen, sofern die Empfängeradresse 0 ist (es also an den Master geht)-
- Vers: Aktuell für SoPC ist die Version 3, auch wenn bereits höhere Versionen in anderen Lehrveranstaltungen eingesetzt wurden. Steht dort eine andere Versionsnummer, muss das Paket verworfen werden.
- Hops: Der Hopcounter zählt, wie oft das Paket weitergeleitet wurde. Er muss um einen hochgezählt werden, bevor weitergeleitet wird, also das Paket an L2_send übergeben wird. Wird eine Antwort gesendet, wird Hops auf 0 zurückgesetzt.

9.3.4 L7_receive

Prototyp: `L7_receive(uint8_t L7_PDU[])`. Isolieren Sie auch hier die 8 Byte große SDU: `uint8_t L7_SDU[])`. Das weitere Verhalten hängt nun von der Applikationsnummer *ApNr* ab. Zwei Grundsätzliche Regeln müssen jedoch unabhängig von der ApNr eingehalten werden:

1. Jedes Paket wird mit **einem** Paket beantwortet; nach dem Motto: **Eins rein, eins raus**. Keine Antwort bedeutet einen Fehler, wie z.B. der Erhalt einer nicht-implementierten ApNr. Mehr als ein Paket als Antwort ist nicht erlaubt. Dies würde zu Engpässen bei den danach kommenden Teilnehmern führen, die die Pakete weiterleiten müssen.
2. Die Antwort muss **so schnell wie möglich** abgesendet werden. Es ist Zeit eine Variable oder einen Port zu lesen und zurück zu gegeben. Verzögerungen an dieser Stelle führen ebenso zu Engpässen.

Daraus folgt, dass Sie hier direkt die Antwortfunktion L7_send aufrufen. Alle zeitintensiven Bearbeitungen, die durch den Empfang eines Pakets ausgelöst werden, sollen durch das Setzen eines Flags in das Hauptprogramm ausgelagert werden. Dort können Sie das Flag testen und nach Bearbeitung zurücksetzen.

Implementieren Sie die folgenden ApNr.:

ApNr 1: set Muster

Der Inhalt der ankommenden SDU von **ApNr 1** stellt das Initial Muster des GoL-Muster in der folgenden Form dar:

ApNr 1:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
GoL-Bit:	63 .. 56	55 .. 48	47 .. 40	39 .. 32	31 .. 24	23 .. 16	15 .. 8	7 .. 0

Initialisieren Sie Ihre GoL-Matrix mit den empfangenen Werten. Schauen Sie sich hierfür Abbildung 7 an. Dabei ist die Matrixposition L(x)(y) = GoL-Bit(n) (mit x=n%8 und y=n/8).

Sende Sie die empfangene Nachricht ohne die SDU zu ändern an den Master zurück.

ApNr 2: NextGen

Der Inhalt der ankommenden SDU von **ApNr 2** wird ignoriert. Wenn **ApNr 2** empfangenen wird, soll eine neue Generation berechnet werden. Das ist das Gleiche Verhalten wie beim Drücken des Tasters. Senden Sie die empfangene Nachricht ohne die SDU zu ändern an den Master zurück.

ApNr 3: read Life

Der Inhalt der ankommenden SDU von **ApNr 3** ist leer.
Das eigenen Muster der GoL soll an den Master zurückgesendet werden.

Füllen Sie hierzu die Zustände ihre 64 GoL-Zellen in die SDU in folgender Form ein:

ApNr 3:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
GoL Bit:	63 .. 56	55 .. 48	47 .. 40	39 .. 32	31 .. 24	23 .. 16	15 .. 8	7 .. 0

Füllen Sie die L7_SDU wie oben dargestellt mit Werten aus Ihrem Speicher. Schauen Sie sich hierfür Abbildung 7 an. Dabei ist GoL-Bit(n) = L(n/8)(n%8). Senden Sie die beschriebene SDU an den Master zurück.

ApNr 4: send color Dead

Der Inhalt der ankommenden SDU von **ApNr 4** stellt den RGB-Wert für tote Zellen in der folgenden Form dar:

ApNr 4:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
Color Bit:	R(7 .. 0)	G(7 .. 0)	B(7 .. 0)	-	-	-	-	-

Initialisieren Sie den RGB-Wert für tote Zellen entsprechend mit den empfangenen Werten und senden Sie die empfangene Nachricht ohne etwas an der L7-SDU zu ändern an den Master zurück.

ApNr 5: send color Life

Der Inhalt der ankommenden SDU von **ApNr 5** stellt den RGB-Wert für lebende Zellen in der folgenden Form dar:

ApNr 5:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
Color Bit:	R(7 .. 0)	G(7 .. 0)	B(7 .. 0)	-	-	-	-	-

Initialisieren Sie den RGB-Wert für lebende Zellen entsprechend mit den empfangenen Werten und sende Sie die empfangene Nachricht ohne etwas an der L7-SDU zu ändern an den Master zurück.

ApNr 6: send Nachbarn

Der Inhalt der ankommenden SDU von **ApNr 6** stellt die Werte der Nachbar-Zellen in der folgenden Form dar:

ApNr 1:	L7_SDU[0]	L7_SDU[1]	L7_SDU[2]	L7_SDU[3]	L7_SDU[4]	L7_SDU[5]	L7_SDU[6]	L7_SDU[7]
N Bit:	-	-	-	'0000' & 35 .. 32	31 .. 24	23 .. 16	15 .. 8	7 .. 0

Die N Bits 0-35 entsprechen den Nachbarn aus Abbildung 7. Initialisieren Sie beim Empfangen der Nachricht die Nachbarzellen in Ihren Speichern und senden Sie die empfangene Nachricht ohne die SDU zu ändern an den Master zurück.

9.3.5 L7_send

Prototyp: `L7_send(uint8_t ApNr, uint8_t L7_SDU[])`. Setzen Sie ApNr und SDU zur Layer 7 PDU (`uint8_t L7_PDU[]`) zusammen. Die übergeben Sie als Layer 3 SDU beim Aufruf an L3_send.

9.3.6 L3_send

Prototyp: `L3_send(uint8_t L3_SDU[])`. Die Layer 3 PCI enthält als Adressat (To-Feld) immer die Adresse 0, da nur an den Master zurückgesendet wird, die Absenderadresse `uint8_t myAddress`, Version 3, und einen auf 0 initialisierten Hopcounter. Kombinieren Sie die 4 Byte PCI mit der SDU zur Layer 3 PDU (`uint8_t L3_PDU[]`). Die übergeben Sie als Layer 2 SDU beim Aufruf an L2_send.

9.3.7 L2_send

Prototyp: `L2_send(uint8_t L2_SDU[])`. Die Layer 2 PCI enthält die CRC Checksumme. Berechnen Sie sie über die SDU und kombinieren SDU und PCI zur `uint8_t L2_PDU[]`, die an L1_send weitergegeben wird.

9.3.8 L1_send

Prototyp: `L1_send(uint8_t L1_SDU[])`. Die Layer 1 PCI besteht aus den beiden Feldern SOF und EOF, die Sie mit 0 füllen. Kombinieren Sie SDU und PCI zur 16 Byte großen `uint8_t L1_PDU[]`. Versenden Sie alle 16 Byte über den UART mit dem Aufruf `neorv32_uart1_putc(L1_SDU[x])`.

9.4 Test des Zugriffs

Testen Sie GoL-GUI mit den Hinweisen und dem Programm. Beides ist im ILU erhältlich: `https://ilu.th-koeln.de/goto.php?target=fold_567972&client_id=thkilu`

Hinweise:

- Fehlermeldung zu 'serial': `python3 pip install pySerial`. Auch einemal deinstallieren (`..pip uninstall..`)/neuinstallieren von *pySerial* kann helfen
- ...

9.5 ISO/OSI Referenzmodell

Das ISO/OSI (International Standard Organisation/ Open Systems Interconnection Model), standardisiert als ISO IEC 7498 bildet eine allgemeine und generische Basis für Netzwerkverbindungen zwischen verschiedenen Systemen.

9.5.1 Schichten, Protokolle, Dienste

Eine erste Einordnung bietet die Aufteilung in Hardware und Software. Bei der Hardware geht es vom Stecker, über Pegelwandler zu Protokollmaschinen. In der Software vom Dekodieren der Daten, über Checksummen, bis zu Routinginformationen. Dies kann auch noch aufgeteilt werden in Betriebssystem-Software und Anwendungssoftware. Abbildung 13 zeigt ein typisches Beispiel eines Mikrocontrollersystems mit integrierter Hardware zur Umsetzung von Netzwerkprotokollen, wie z.B. ein Ethernet MAC (Medium Access Controller). Stecker und Pegelwandler sind Teil der Netzwerk Infrastruktur außerhalb des Chips. Das Betriebssystem enthält Treiber, die die entsprechende Netzwerk-Hardware ansprechen und verwalten können. Darauf können Kommunikationsdienste aufsetzen, die die Hardware nutzen, um bestimmte Dienste anzubieten, wie z.B. Senden/Empfangen, Weiterleiten (Routing), Fehlerbehandlung. Das Anwendungsprogramm kommuniziert über die Betriebssystem-Schnittstellen, um Dienste zu nutzen.

Die Abbildung 14 zeigt die Zusammenhänge zwischen *Schichten, Protokollen und Schnittstellen* im Kommunikationsmodell:

- Schicht: Abstraktion der Kommunikation für eine Verarbeitungsebene, wie z.B. Kommunikation auf elektrischer Ebene, Kommunikation auf Ebene von Bitpaketen, .., Kommunikation auf Ebene einer Anwendung
- Protokoll: Menge von Regeln, die die Kommunikation zwischen den Elementen der gleichen Schicht regelt. Beispiel: Protokoll zwischen den Netzzugangsschnittstellen (Bitübertragung, Senderecht), Protokoll zwischen Anwendungsprozessen über "Prozeduraufrufe"(RPC-remote procedure call)
- Schnittstelle: Die Schnittstelle beschreibt, wie eine untere Schicht ihre Dienste der darüberliegenden bereitstellt

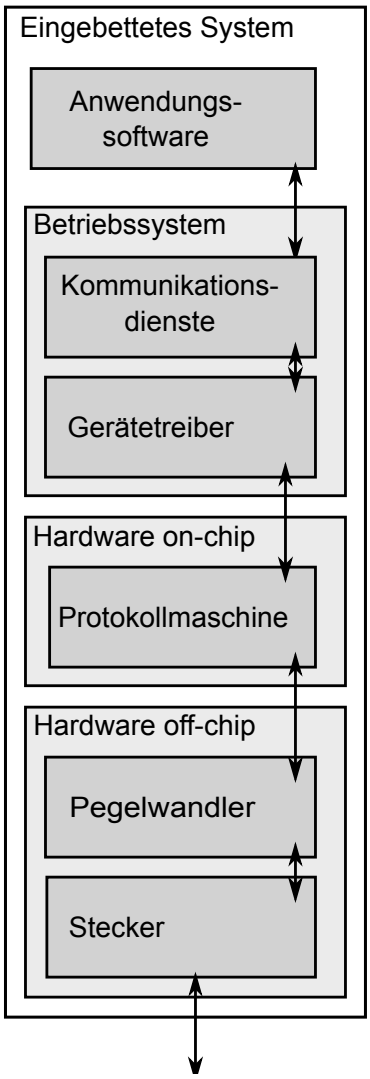


Abbildung 13: Beispiel beteiligter Hard- und Software

9.5.2 PDU, SDU, PCI, ICI

Beim Versenden von Nachrichten werden die Daten von einer höheren Schicht (N+2) zur darunterliegenden Schicht (N+1) weitergegeben: Abbildung 15 zeigt, wie die (N+2)-PDU (Protocol Data Unit) an die darunter liegenden Schicht (N+1) weitergegeben wird und dort zur (N+1)-SDU (Service Data Unit) wird.

Die (N+1)-Schicht ergänzt weitere vom Protokoll vorgeschriebenen Kontrollinformationen: die (N+1)-PCI (Protocol Control Information). SDU und PCI zusammen stellen die PDU der N+1 Schicht dar, die an die darunter liegende N-Schicht als (N)-SDU weitergegeben wird. Hier werden die Protokollinformationen (N)-PCI der N-Schicht hinzugefügt, usw. So fügt jede Schicht Protokollinformationen hinzu, die für die jeweilige gleiche Schicht im Empfänger wichtige Informationen enthält.

Auf der Empfängerseite werden die Daten von Schicht zu Schicht weiter nach oben gegeben. Die entsprechenden Protokollinformationen (PCI) werden von der PDU getrennt und interpretiert, und die SDU weiter nach oben gereicht.

Es gibt weitere Kontrollinformationen, wie z.B. Schnittstellenparameter, die nicht mit in die Nachricht eingebaut werden und auch nicht weitergereicht werden, aber von einer Schicht zur anderen übertragen wird, z.B. die Baudrate einer seriellen Schnittstelle. Dies sind Interface Control Information (ICI).

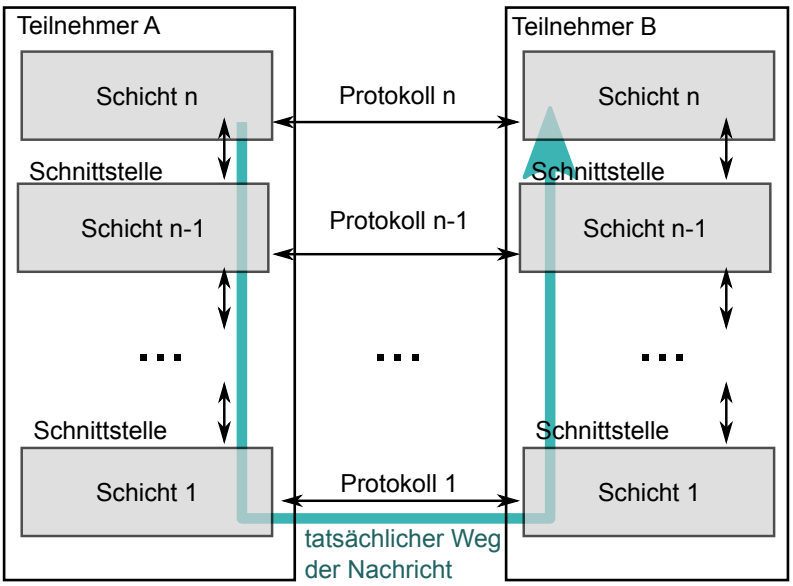


Abbildung 14: Schichten, Protokolle, Schnittstellen

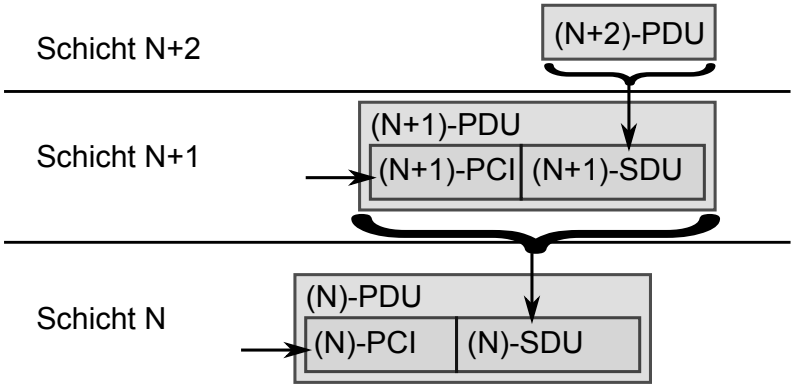


Abbildung 15: Zusammenhang zwischen PDU, SDU und PCI der Schichten

9.5.3 Die OSI-Schichten

Das ISO/OSI Referenzmodell beschreibt Schichten. Es müssen nicht alle Schichten genutzt werden und eine Unterteilung einer Schicht in weitere, einzelne Schichten ist möglich. Beispielsweise wird in CAN nur Schicht 1 und Schicht 2 genutzt, diese aber wiederum in bis zu drei weitere unterteilt (vergleiche dazu Kapitel **??**).

- **Schicht 7: Anwendungsschicht (Application Layer):** Anwendungsorientierte Dienste, z.B. Zugriff auf einen Anwendungswert ohne Betrachtung des Netzwerks. Die Anwendungsschicht stellt Funktionen für die Anwendungen zur Verfügung.
- **Schicht 6: Darstellungsschicht (Presentation Layer), Präsentationsschicht:** Informationen müssen in standardisierte Formate übertragen werden, z.B. wegen verschiedener Darstellungen von Zahlen in Rechnern; Kryptographie für Übertragung sensibler Daten; Übertragung von Nachrichten-Strukturinformationen, z.B. XDR (eXternal Data Representation, ein Format zur Datenserialisierung, um Daten zwischen Systemen mit unterschiedlicher Zahlenrepräsentation, wie little/big endian auszutauschen)
- **Schicht 5: Sitzungsschicht (Session Layer), Verbindungsschicht:** Aufbau (und Abbau) einer virtuellen Kommunikationsverbindung (Punkt-zu-Punkt-Verbindung), z.B. eine Verbindung, von einem Terminalprogramm eines Rechners zum Rechner, auf dem das Terminal arbeitet, z.B. RPC (Remote Procedure Call)
- **Schicht 4 Transportschicht (Transport Layer):** Senden von Einzelnachrichten von einer Station zu einer anderen (virtuelle Punkt-zu-Punkt-Übertragung von Einzelnachrichten) durch Aufteilung in Pakete, mehrfaches Senden von Paketen bei Fehlern, z.B TCP/UDP

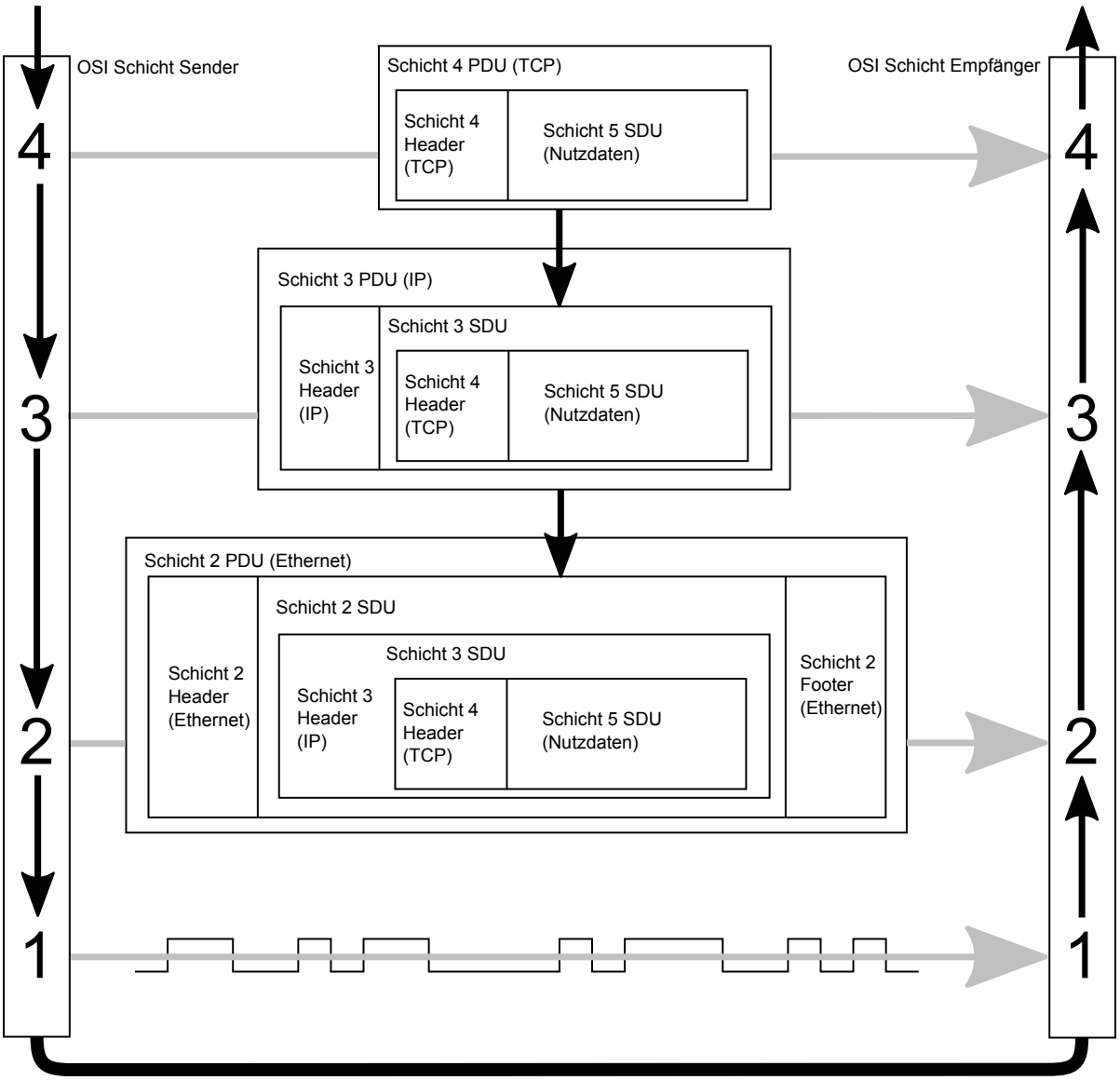


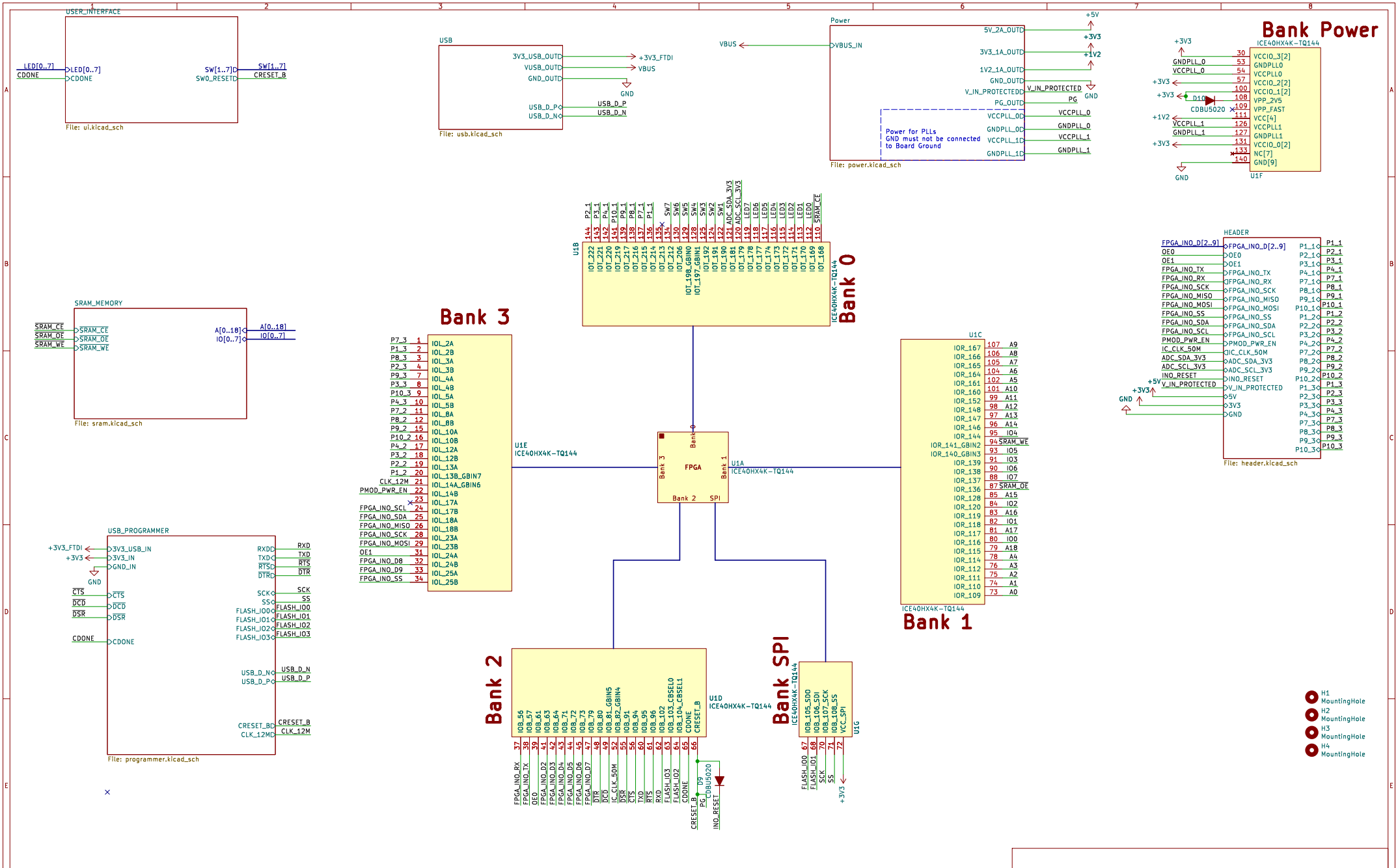
Abbildung 16: TCP/IP über Ethernet, Darstellung eines Teils der OSI Layer

- **Schicht 3 Vermittlungsschicht (Network Layer):** Wegewahl im Netz, Routing, z.B. IP
- **Schicht 2 Sicherungsschicht (Data Link Layer):** Übermittlung von Station zu Station als sichere Verbindung, Zugangskontrolle zum Medium, z.B. Ethernet 802.3 MAC, Prüfsummenberechnung, Datenflusskontrolle
- **Schicht 1 Bitübertragungsschicht (Physical Layer):** Physikalische Übertragung, Repräsentation von Bits/Bitgruppen, z.B. Ethernet PHY, CAN Pegelwandler (PMA), Stecker, Antenne

Abbildung 16 zeigt eine Einordnung von TCP/IP über Ethernet in die Schichten des OSI Referenzmodells. Die Einordnung ist nicht eindeutig und findet sich in manchen Quellen anders, z.B. wird TCP auch wegen der Verbindungsorientierung in Schicht 5 angesehen. Die Abbildung illustriert die Weitergabe der SDUs durch die verschiedenen Schichten. Die Header und Footer der verschiedenen Protokolle TCP, IP und Ethernet stellen die entsprechenden PCI der Schichten dar.

A Iceduino Schaltplan

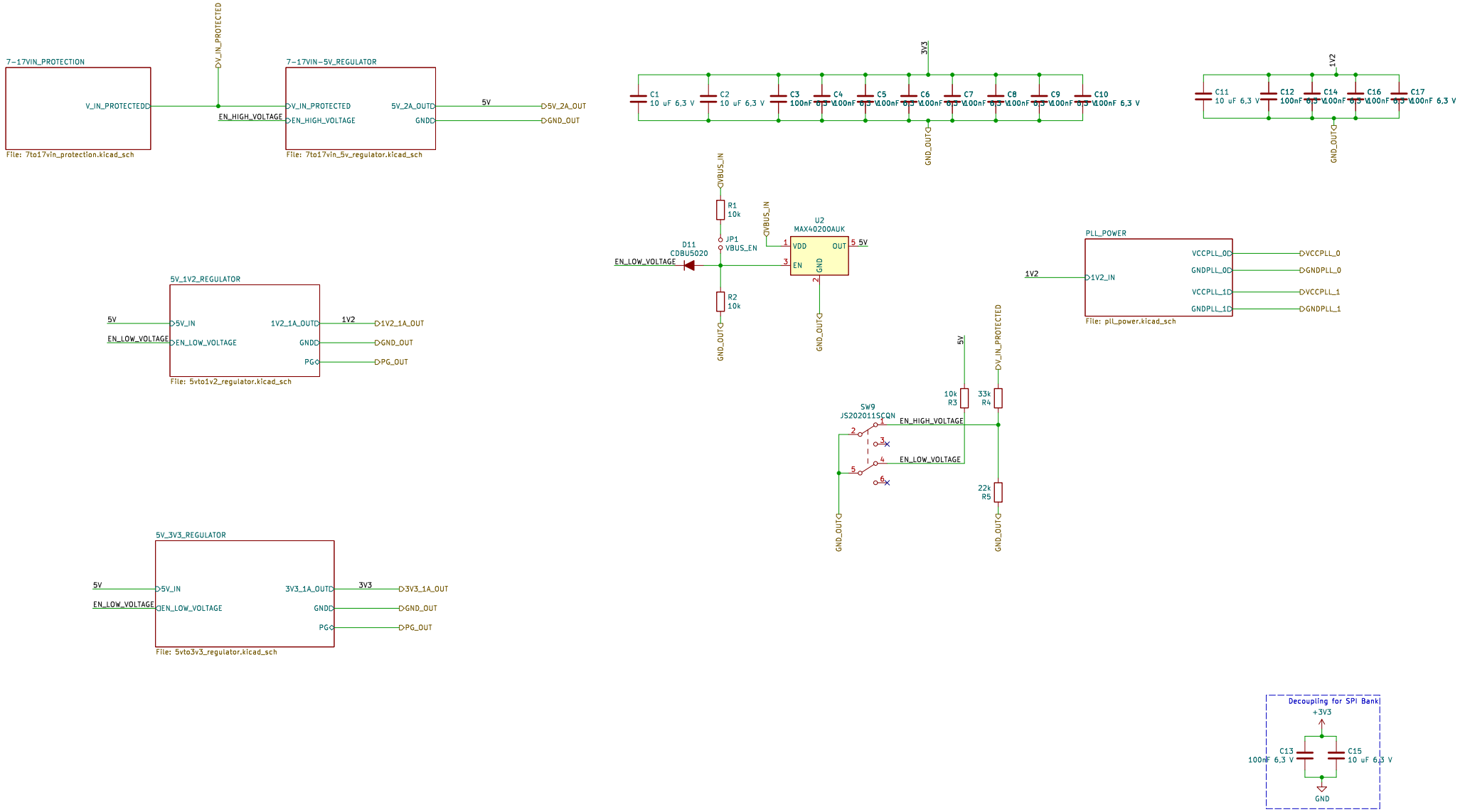
- Seite 44: Iceduino Schaltplan Ubersicht
- Seite 45: Power
- Seite 46: 7-17V input Protection
- Seite 47: 7-17V input 5V Regulator
- Seite 48: 5V zu 1.2 V Regulator
- Seite 49: 5V zu 3.3 V Regulator
- Seite 50: PLL Stromversorgung
- Seite 51: LEDs Taster Schalter
- Seite 52: SRAM
- Seite 53: USB Anschluss
- Seite 54: Level Shifter Header 50 MHz
- Seite 55: Arduino Header
- Seite 56: PMOD Header
- Seite 57: FTDI Programming Interface
- Seite 58: Iceduino Platine
- Seite 58: Kupfer Bestückungsseite
- Seite 59: Bestückungsdruck



Sheet: /
File: iceduino.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

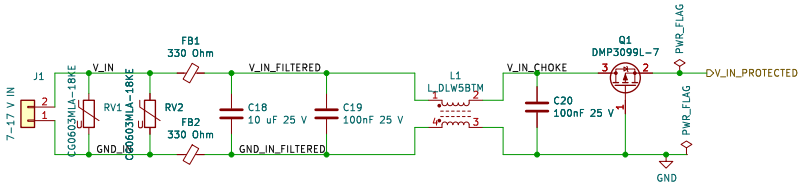
Size: A3	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 1/14



Sheet: /Power/
File: power.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

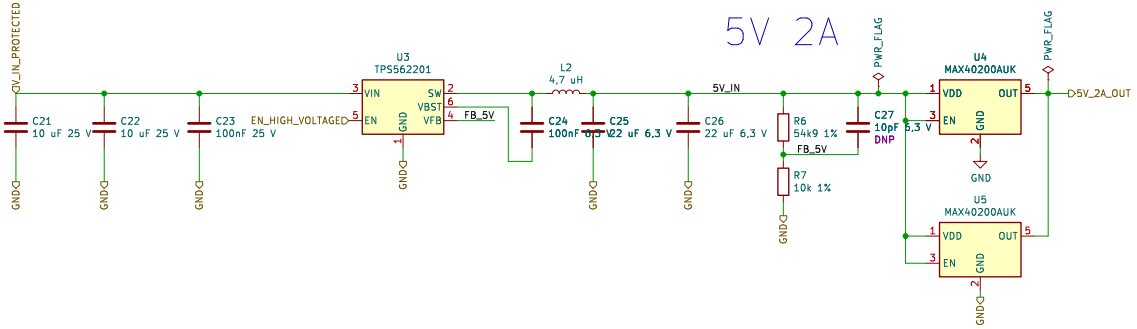
Size: A3	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 2/14



Sheet: /Power/7-17VIN_PROTECTION/
File: 7to17vin_protection.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 3/14

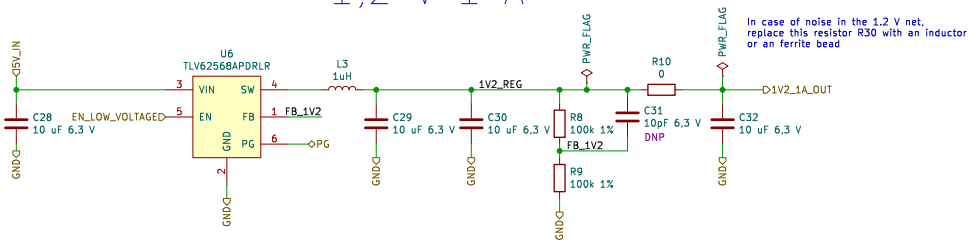


Sheet: /Power/7-17VIN-5V_REGULATOR/
File: 7to17vin_5v_regulator.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 4/14

1,2 V 1 A

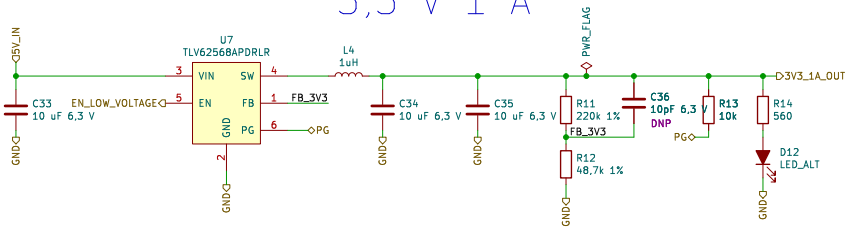


Sheet: /Power/5V_1V2_REGULATOR/
File: 5vto1v2_regulator.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 5/14

3,3 V 1 A

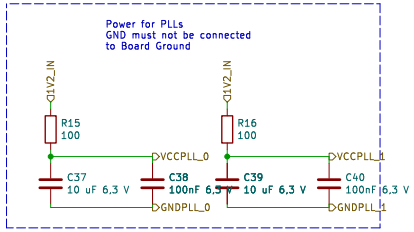


Note: Use TLV62568ADRLR for 2 A

Sheet: /Power/5V_3V3_REGULATOR/
File: 5vto3v3_regulator.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

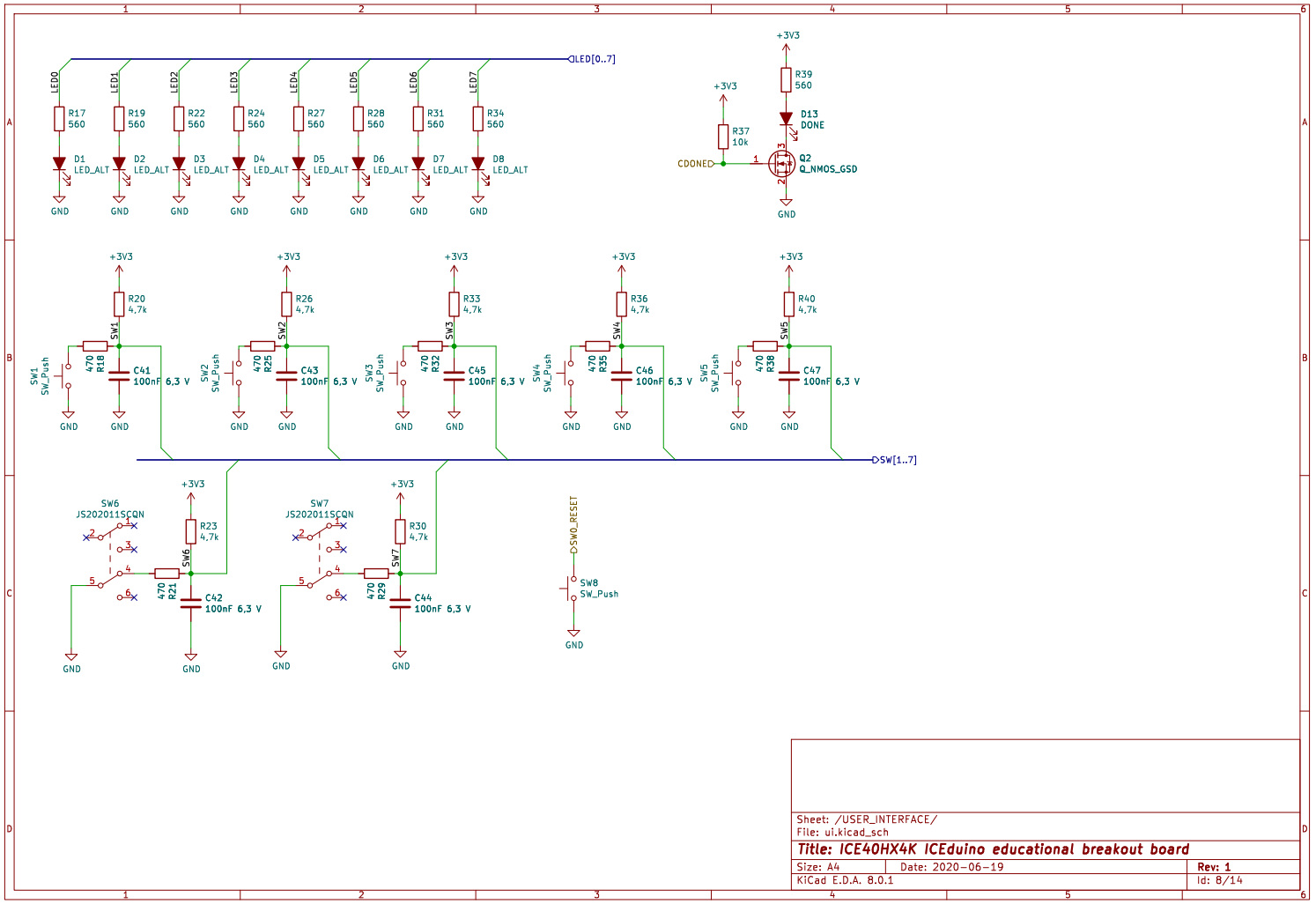
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 6/14



Sheet: /Power/PLL_POWER/
File: pll_power.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

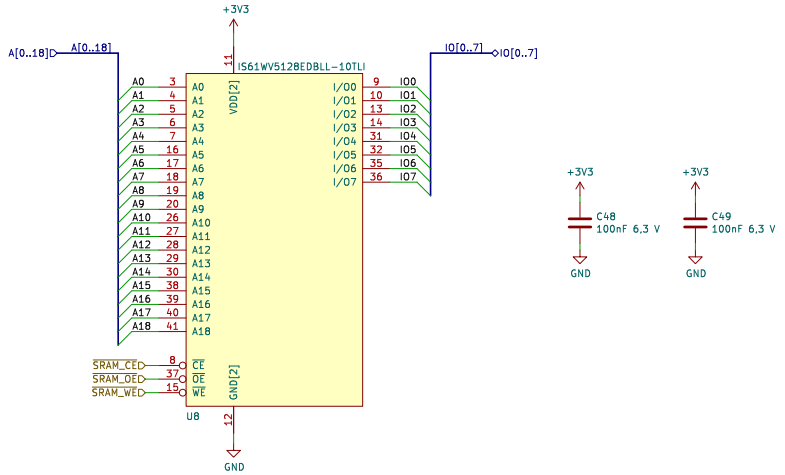
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 7/14



Sheet: /USER_INTERFACE/
File: ui.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

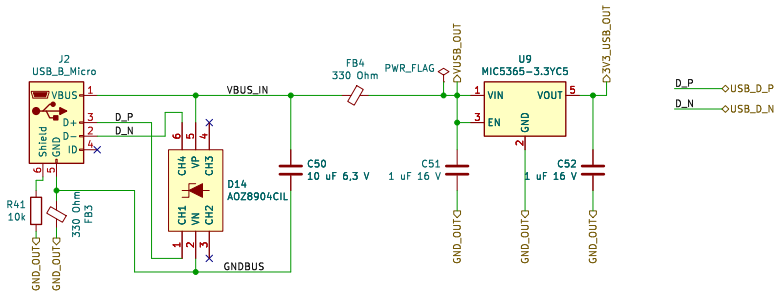
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 8/14



Sheet: /SRAM_MEMORY/
File: sram.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

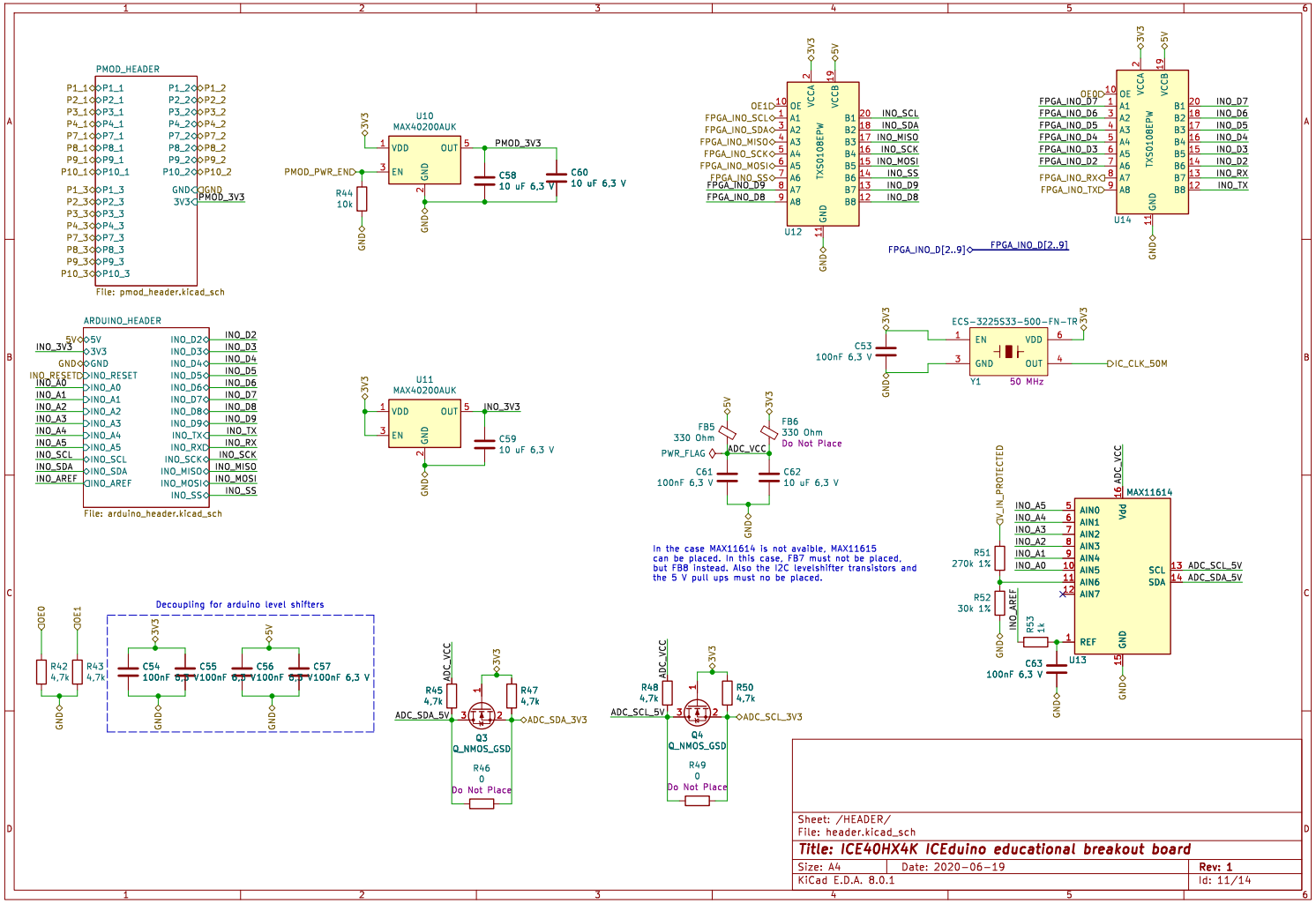
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 9/14



Sheet: /USB/
File: usb.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

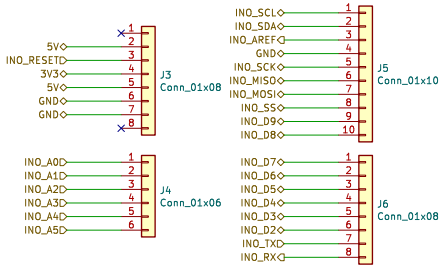
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 10/14



Sheet: /HEADER/
File: header.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

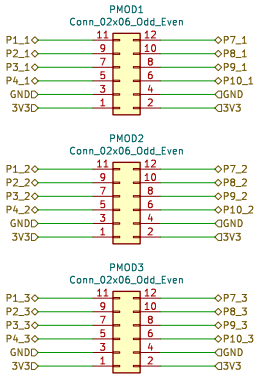
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 11/14



Sheet: /HEADER/ARDUINO_HEADER/
File: arduino_header.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

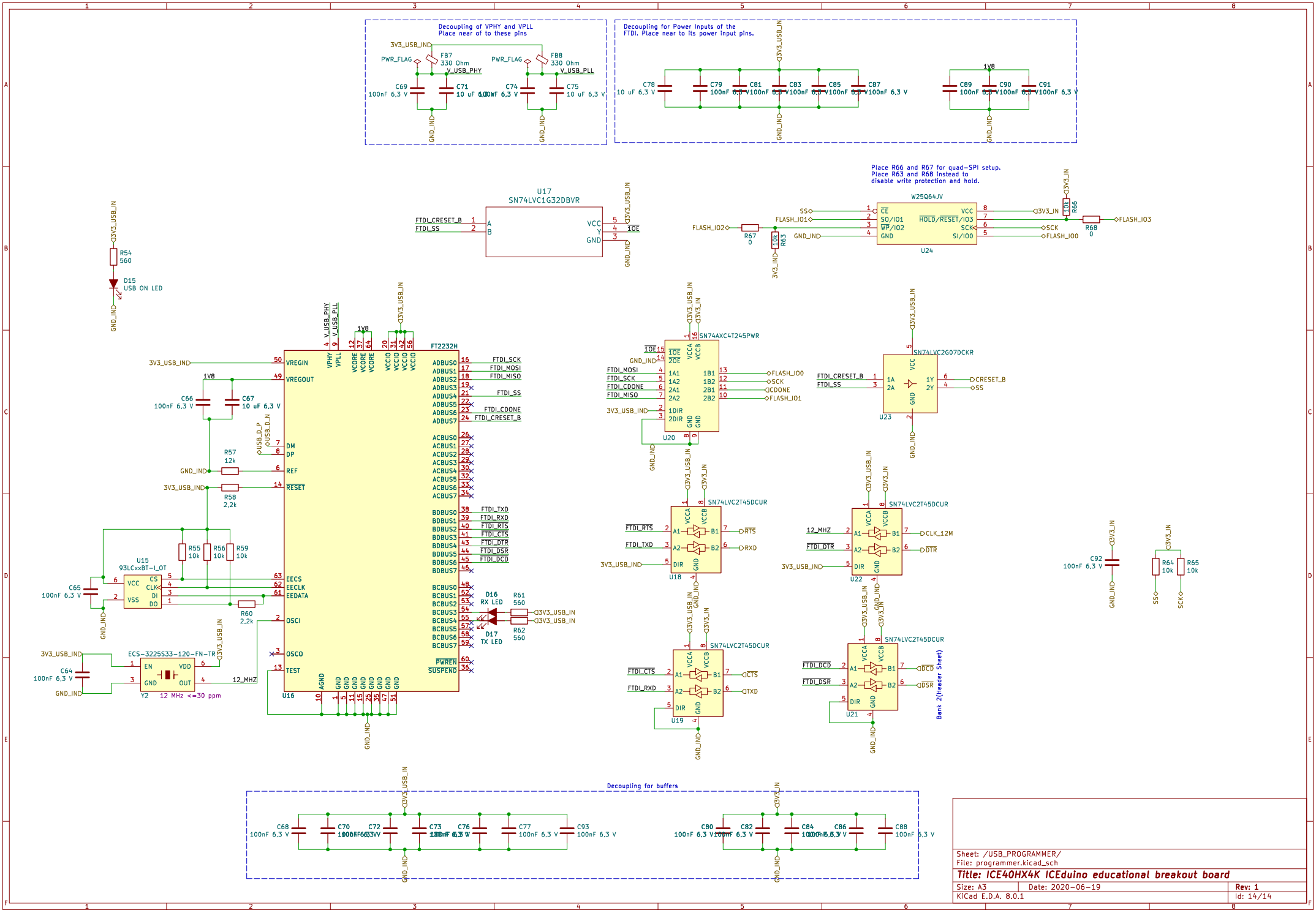
Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 12/14

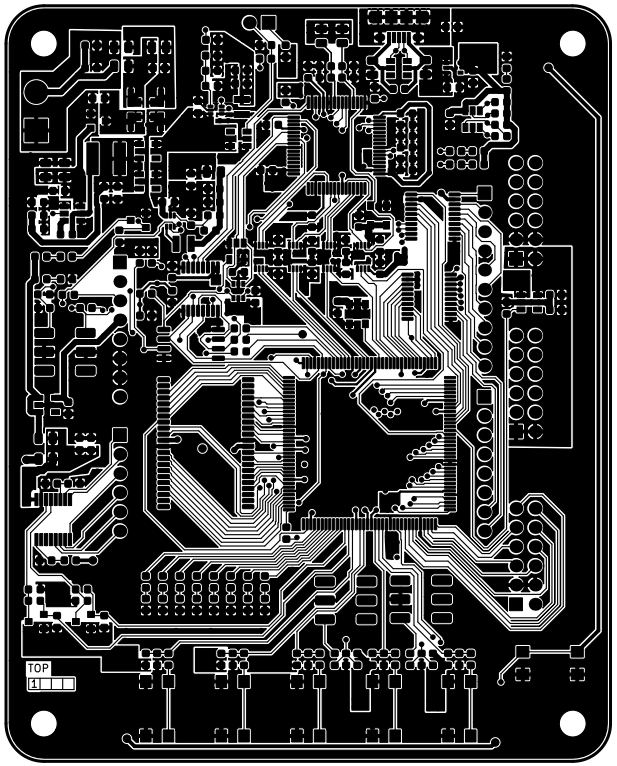


Sheet: /HEADER/PMOD_HEADER/
File: pmod_header.kicad_sch

Title: ICE40HX4K ICEduino educational breakout board

Size: A4	Date: 2020-06-19	Rev: 1
KiCad E.D.A. 8.0.1		Id: 13/14

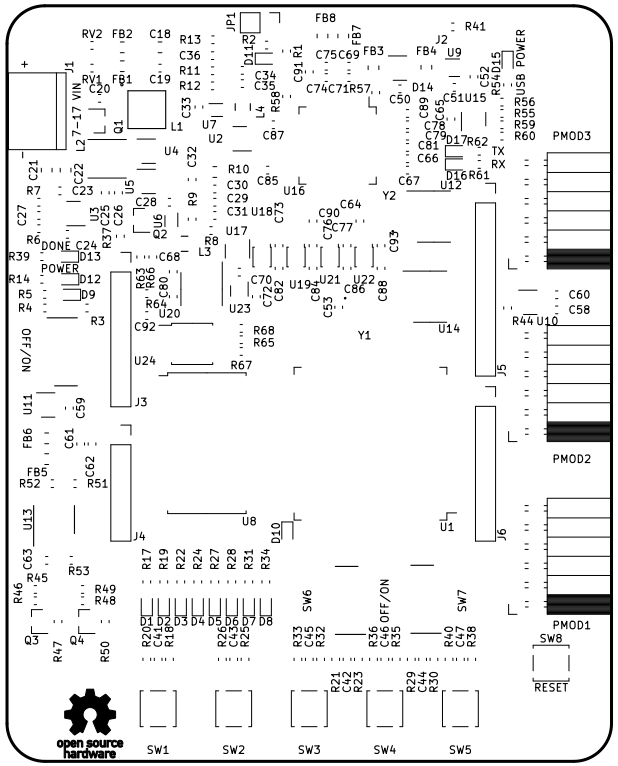

Decoupling of VPHY and VPLL
Place near of to these pins
Decoupling for Power Inputs of the
FTDI. Place near to its power input pins.
Place R66 and R67 for quad-SPI setup.
Place R63 and R68 instead to
disable write protection and hold.
U17
SN74LVC1G32DBVR
W25Q64JV
U24
FT2232H
U16
SN74AXC4T245PWR
U20
SN74LVC2G07DCKR
U23
SN74LVC2T45DCUR
U18
U19
U21
U22
93LCxxBT-I_OT
U15
ECS-3225S33-120-FN-TR
Y2
12 MHz <=30 ppm
D15
USB ON LED
D16
RX LED
D17
TX LED
Decoupling for buffers
Bank 2(Header Sheet)
Sheet: /USB_PROGRAMMER/
File: programmer.kicad_sch
Title: ICE40HX4K ICEduino educational breakout board
Size: A3
Date: 2020-06-19
Rev: 1
KiCad E.D.A. 8.0.1
Id: 14/14



Sheet:
File: iceduino.kicad_pcb

Title:

Size: A4	Date:	Rev:
KiCad E.D.A. kicad (5.1.6)-1		Id: 1/1



Sheet:
File: iceduino.kicad_pcb

Title:

Size: A4	Date:	**Rev:**
KiCad E.D.A. kicad (5.1.6)-1		Id: 1/1

B Hello Blinky

Listing 16: VHDL Hello Blinky

```
library ieee;
use ieee.std_logic_1164.all;

entity hello_blinky is
  port (
    clk   : in  std_logic;
    reset : in  std_logic;
    led   : out std_logic);
end hello_blinky;

architecture behv of hello_blinky is
  signal cnt : integer range 0 to 100;
  signal q     : std_logic;

begin

  led <= q;

  -- purpose: Counter als Teiler
  -- type   : sequential
  -- inputs : clk, reset
  -- outputs: q
  cnt_p : process (clk) is
  begin  -- process cnt_p
    if rising_edge(clk) then            -- rising clock edge
      if reset = '1' then               -- synchronous reset (active high)
        q   <= '0';
        cnt <= 0;
      else
        if cnt = 100 then
          cnt <= 0;
          q <= not q;
        else
          cnt <= cnt + 1;
          q   <= q;
        end if;
      end if;
    end if;
  end process cnt_p;
end behv;

```

Listing 17: Makefile Simulation

```
# vhdl files
TESTBENCH=hello_blinky_tb
FILES = ../rtl/*vhd
VHDLEX = .vhd

# testbench
TESTBENCHFILE = ./${TESTBENCH}$(VHDLEX)


#GHDL CONFIG
GHDL_CMD = ghdl
GHDL_FLAGS  =

SIMDIR = simulation
STOP_TIME = 10us
GHDL_SIM_OPT = --stop-time=$(STOP_TIME) --stop-delta=100000

WAVEFORM_VIEWER = gtkwave

.PHONY: clean

all: clean compile run view

compile:
        @mkdir -p $(SIMDIR)
        @$(GHDL_CMD) -i $(GHDL_FLAGS) --workdir=$(SIMDIR) --work=work $(TESTBENCHFILE) $(FILES)
        @$(GHDL_CMD) -m $(GHDL_FLAGS) --workdir=$(SIMDIR) --work=work $(TESTBENCH)


```

```
run:
	@$(GHDL_CMD) -r --workdir=$(SIMDIR) --work=work $(TESTBENCH) --wave=$(SIMDIR)/$(TESTBENCH).ghw  $(GHDL_SIM_OPT)

view:
	$(WAVEFORM_VIEWER) --dump=$(SIMDIR)/$(TESTBENCH).ghw --save ./$(TESTBENCH).gtkw

clean:
	@rm -rf $(SIMDIR)
```

Listing 18: VHDL Hello Blinky Testbench mit Asserts

```
-------------------------------------------------------------------------------
-- Title      : Testbench for design "hello_blinky"
-- Project    :
-------------------------------------------------------------------------------
-- File       : hello_blinky_tb.vhd
-- Author     : thk  <thk@thkvm>
-- Company    :
-- Created    : 2022-03-30
-- Last update: 2022-03-30
-- Platform   :
-- Standard   : VHDL'93/02
-------------------------------------------------------------------------------
-- Description:
-------------------------------------------------------------------------------
-- Copyright (c) 2022
-------------------------------------------------------------------------------
-- Revisions  :
-- Date        Version  Author  Description
-- 2022-03-30  1.0      thk     Created
-------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

-------------------------------------------------------------------------------

entity hello_blinky_tb is

end entity hello_blinky_tb;

-------------------------------------------------------------------------------

architecture dut of hello_blinky_tb is

  -- component ports
  signal clk   : std_logic := '0';
  signal reset : std_logic;
  signal led   : std_logic;


begin  -- architecture dut

  -- component instantiation
  DUT : entity work.hello_blinky
    port map (
      clk   => clk,
      reset => reset,
      led   => led);

  -- clock generation
  Clk <= not Clk after 10 ns;

  -- waveform generation
  WaveGen_Proc : process
  begin
    reset <= '0'; wait for 87 ns;
    reset <= '1'; wait for 41 ns;
    reset <= '0';
    report "Reset 0, geht los";
    wait for 100*20 ns - 10 ns;

    report "Test beginnt";
    assert (led = '0') report "LED ist nicht 0: falsch" severity warning;
    assert (led /= '0') report "LED ist 0: ok" severity note;
    wait for 20 ns;
    assert (led = '1') report "LED ist nicht 1: falsch" severity warning;
    assert (led /= '1') report "LED ist 1: ok" severity note;
```

```


    wait;
  end process WaveGen_Proc;



end architecture dut;

-------------------------------------------------------------------------------

configuration hello_blinky_tb_dut_cfg of hello_blinky_tb is
  for dut
  end for;
end hello_blinky_tb_dut_cfg;

-------------------------------------------------------------------------------
```

C Schaltplan Multi-Iceduino

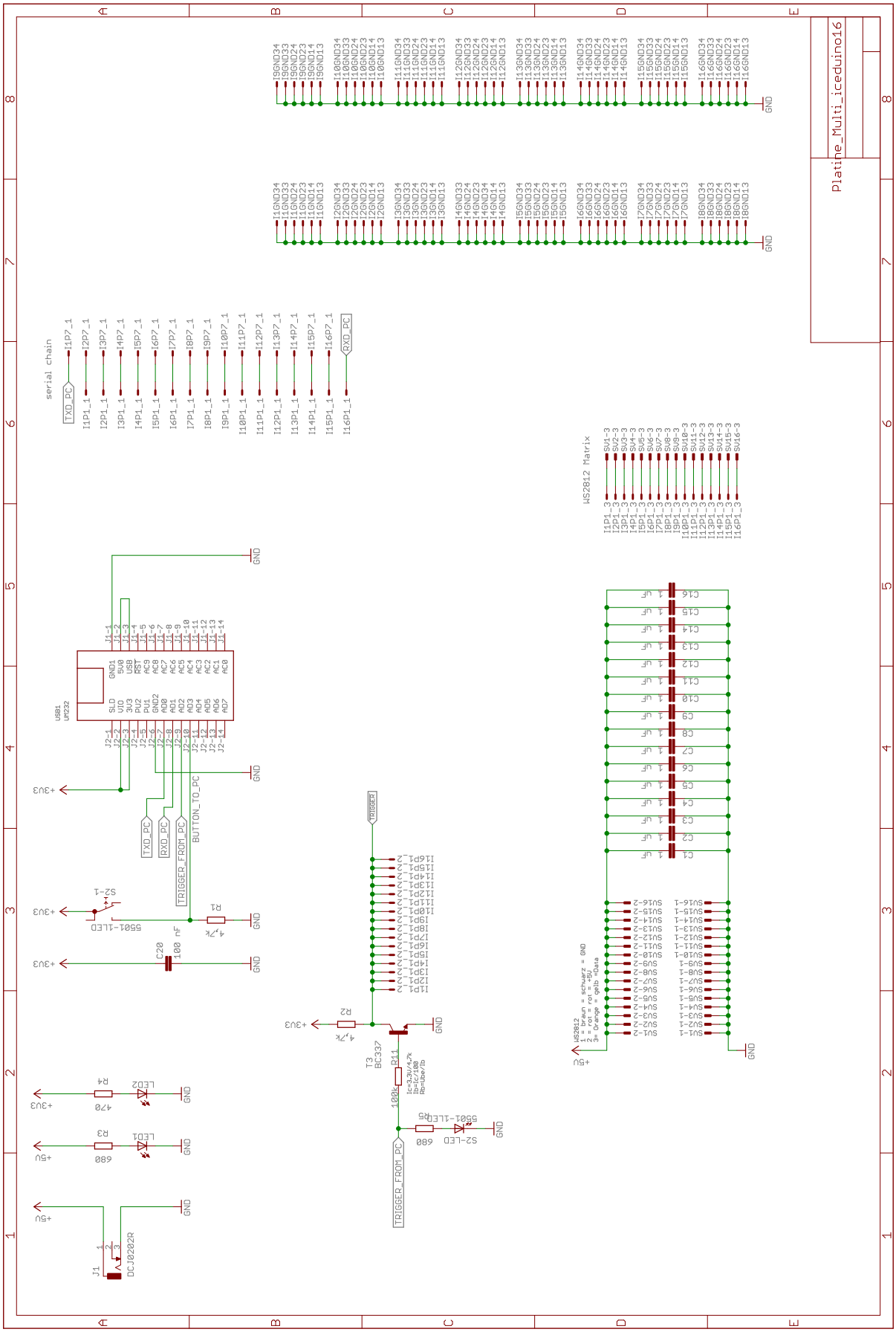


Abbildung 17: Schaltplan des Base-Boards für mehrere Iceduinos

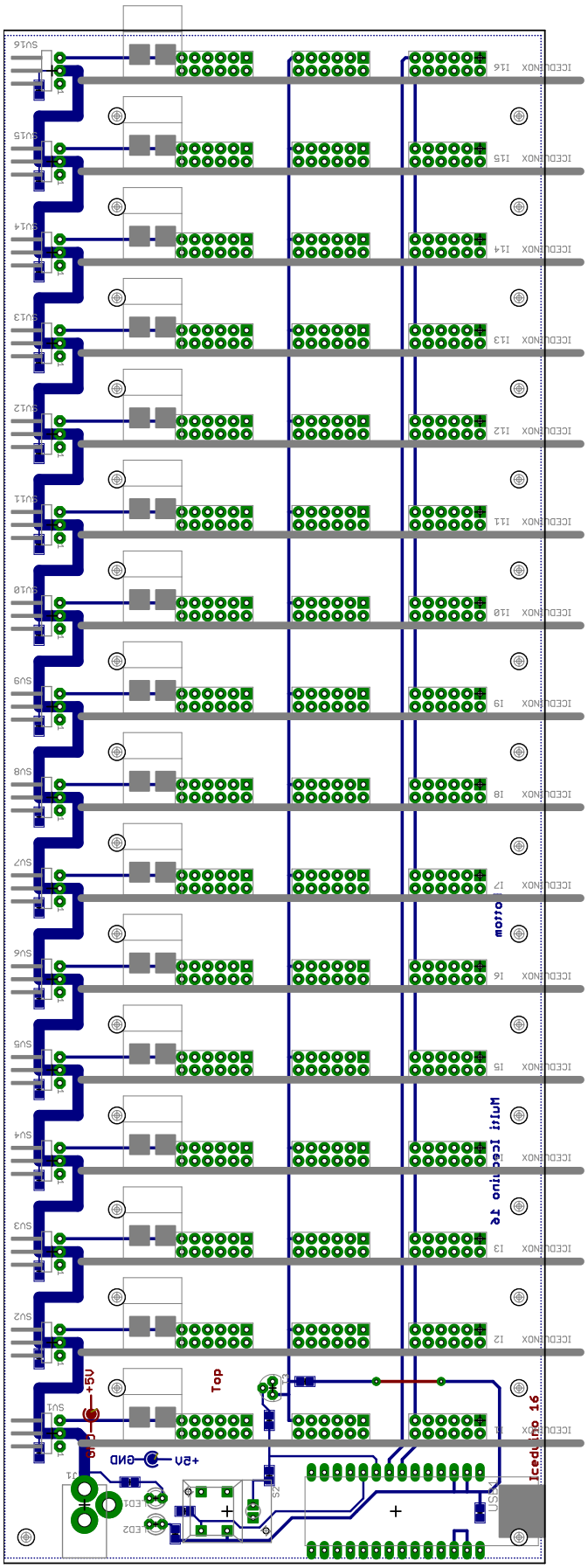


Abbildung 18: Bestückungsplan des Base-Boards für mehrere Iceduinos